

UTRECHT UNIVERSITY
Computing Science



MASTER'S THESIS

Powerful Graph Neural Networks for Money Laundering Pattern Detection

Author:
Stan VERLAAN
ICA-6813526

Supervisors:
Dr. E.J. VAN LEEUWEN
Dr. I.R. KARNSTEDT-HULPUS

Second examiner:
Prof. dr. H.L. BODLAENDER

*A thesis submitted in fulfillment of the requirements
for the degree of Master of Science*

in the

Faculty of Science
Department of Information and Computing Science

July 12, 2024

Acknowledgements

I would like to express my deepest gratitude to my supervisors, dr. Erik Jan van Leeuwen and dr. Ioana Karnstedt-Hulpus, for their invaluable guidance throughout the course of my master's thesis. Their expertise, insights, and constructive feedback have been instrumental in shaping this work. Additionally, I would like to thank prof. dr. Hans Bodlaender for reading and judging my thesis. I am also thankful to Vahid, Ramón and Karen for our bi-weekly group sessions and lunches, where I could share my progress, receive helpful suggestions, and draw inspiration from their work on similar projects. I extend my gratitude to my fellow students and good friends for the fun we have had over the past five years and to my parents and brother for always supporting me. Lastly, I want to express my sincerest appreciation to Lianne for her help, belief and encouragement throughout this project. You have the best patience and always provide me with much-needed distraction during the evenings and weekends. Thank you.

Abstract

Powerful Graph Neural Networks for Money Laundering Pattern Detection

by Stan VERLAAN

This research enhances graph neural networks (GNN) for identifying suspicious accounts involved in money laundering. Extending the work of Egressy et al. (2024), we modify and propose a set of techniques that enable GNNs to detect suspicious subgraph patterns in the weighted temporal networks underlying financial data. These techniques include a novel message passing scheme, which allows for the indication of edge directionality within a single aggregator function, element-wise edge weight multiplication, and an LSTM aggregator that can learn from the sequential order of edges imposed by timestamps. Our LAS-GNN model is based on an inductive learning framework and can generalize across different networks. Experimental results on synthetic networks show that LAS-GNN is robust and can identify basic money laundering patterns, such as smurfing motifs and simple cycles up to length 6, to near perfection, outperforming a graph isomorphism network benchmark with edge features. While the primary focus is on the financial crime domain, the findings have broader applications in dynamic network settings.

Contents

Acknowledgements	i
Abstract	ii
1 Introduction	1
1.1 Background	1
1.2 Our Contributions	2
2 The State of Money Laundering Detection	4
2.1 Transaction-Based Machine Learning for AML	4
2.1.1 Supervised Learning	5
2.1.2 Unsupervised Learning	6
2.2 Graph-based Approaches for AML	7
2.2.1 Domain-led	7
2.2.2 Graph-based Features	8
2.2.3 Random Walk Embeddings	10
2.2.4 Deep Learning	10
3 Graph Neural Networks	13
3.1 Transductive vs. Inductive	13
3.2 Graph Convolutional Network	14
3.3 Inductive Representation Learning	15
3.3.1 GraphSAGE	16
3.3.2 Graph Isomorphism Network	18
3.4 GNN Techniques for Graph Pattern Finding	18
3.4.1 Modelling Node Identity	18
3.4.2 Modelling Edge Directionality	19
3.4.3 Modelling Weights	20
3.4.4 Modelling Time	20
4 Method	21
4.1 Money Laundering Patterns	21
4.2 Modelling Edge Directionality: Signed Message Passing	22
4.3 Modelling Weights: Edge Weight Multiplication	23
4.4 Modelling Time: LSTM Aggregation	23
4.4.1 Long Short-Term Memory	24
4.4.2 LSTM Aggregator	25
4.5 The LAS-GNN Model	25
4.6 Implementation	27
4.6.1 Ego IDs	27
4.6.2 Signed Message Passing	28

5	Experimental Setup	29
5.1	Synthetic Network Generation	29
5.1.1	Directed Watts-Strogatz Model	29
5.1.2	Network Parameter Settings	31
5.1.3	Edge Weights and Timestamps	32
5.1.4	Ground Truth	33
5.2	Model Training	33
5.3	Compared Models	34
5.4	Evaluation Metrics	35
5.4.1	Confusion Matrix	35
5.4.2	Accuracy, Precision, Recall, F_1 -score	35
5.4.3	Precision-Recall Curve, Average Precision	36
5.5	Technical Specifications	37
6	Results	38
6.1	Preliminary Experiments	38
6.1.1	Edge Directionality	38
6.1.2	Number of GNN Layers	39
6.1.3	Number of LSTM layers	40
6.2	Unweighted Non-temporal Detection	42
6.3	Weighted Non-temporal Detection	42
6.3.1	Edge Weight Similarity Task	44
6.4	Unweighted Temporal Detection	45
6.5	Weighted Temporal Detection	46
6.6	Longer Cycle Detection	49
7	Discussion	51
7.1	Discussion of Results	51
7.2	Experimental Limitations	52
7.3	Model Limitations	52
8	Conclusion	53
	Bibliography	54
A	Weighted Non-temporal Detection	60
A.1	Gather-scatter	60
A.2	C3	61
B	Weighted Temporal Detection	62
B.1	Gather-scatter	63
B.2	Scatter-gather	64
B.3	C3	65

Chapter 1

Introduction

1.1 Background

Money laundering (ML) is the process of concealing the origin of illegitimately obtained money. It involves the conversion of illicit funds into seemingly legitimate assets. This “dirty” money can be obtained from all sorts of illegal activities, such as embezzlement, drug and human trafficking, and the funding of terrorist organizations, and is disguised so that it can be used without the detection of said activities. According to the United Nations Office on Drugs and Crime ([UNODC](#)), between 2% to 5% of the global Gross Domestic Product (GDP) can be attributed to money laundering, which amounts to 800 billion to 2 trillion US dollars. This makes the money laundering business one of the largest industries in the world. With the growth of global economy, e-businesses and online transactions, it is predicted that the number of money laundering crimes will continue to increase ([Zhang et al., 2003](#)). The implications this has for both the economy and society as a whole are significant and wide-ranging. Aimless movement of large funds can have destabilizing effects on financial systems, and money laundering often goes hand-in-hand with tax and duty evasion ([Capgemini](#)). Furthermore, these funds facilitate criminal and terrorist organizations, with obvious consequences for national safety.

Given these negative effects, detecting money laundering processes is of paramount importance. Anti-Money Laundering (AML) refers to the set of laws, regulations, and procedures designed and aimed at detecting, preventing, and prosecuting individuals and entities involved in money laundering. Banks monitor transactions and report suspicious activity based on a set of rules that are predefined by domain experts. Such rules and the statistical thresholds related to different scenarios are motivated by industry red flags (transactions to suspicious countries) and basic statistical indicators (amounts over a certain threshold), as well as expert judgement ([Rompaye](#)). Even though this seems like a sensible and straightforward way of translating money laundering concerns into a working system, such rule-based approaches suffer from some drawbacks ([Gao et al., 2009](#)). Sophisticated criminals who are familiar with the rules can easily deceive the system and evade detection. In turn, these rule-based systems need to be updated constantly to keep up with the ever-evolving money laundering strategies. Furthermore, static rules may not always be able to capture the complex patterns of money laundering trails. While the scope of a single assessment can be broadened to include numerous transactions, this may result in much more intricate rules that are prone to implementation errors and whose effectiveness are hard to discern.

A valuable method used by transaction offices and advocated to banks by De Nederlandse Bank (DNB) for detecting money laundering processes is the five-step process scheme of topologies ([De Nederlandse Bank](#)). This method analyzes historical data of transactions and clients based on established money laundering topologies to identify accounts whose activities are similar to the accounts in such topologies. The newly detected accounts may reveal to be part of novel money laundering topologies, which in turn can be employed to analyze new historical data. However, when rule-based approaches are used for matching targets and analyzing the results, the same drawbacks still apply. Moreover, they are often associated with the generation of a

large number of false-positives (Gao et al., 2009) and the manual follow-up of the alerts can be a time-consuming and labour-intensive process.

To aid in this process, more and more institutions consider the use of machine learning. With machine learning, advanced algorithms can be utilized to identify suspicious money laundering cases directly from the data. While this can be much more efficient, it also turns out to be more precise and effective in practice: KPMG Belgium’s experience at a leading bank showed that their identification of suspicious activity can be improved by up to 40% by replacing rule-based and scenario-based tools with machine learning models (Rompaye). Apart from the increase in performance, changing to AI-based solutions can help overcome some limitations of traditional systems, since they are suitable for adapting to new money laundering schemes and can meet the demand of the high volume of transactions performed on a daily basis.

In literature, a range of different machine learning models are proposed and their potential for the domain of financial crime is investigated. These comprise anomaly detection algorithms and traditional supervised learning approaches that can handle the severe data imbalance related to such tasks. Recently, more sophisticated deep learning techniques that utilize the underlying network structure of transaction data have emerged. Graph Neural Networks (GNNs) that operate on graph-structured data can be deployed for classifying accounts suspected of money laundering. Nevertheless, research into the capabilities of such models for AML is still in its early stages and often focus on extensive features while ignoring the structural pattern detection capabilities necessary in the following task: *Given one or more graph(s) and multiple examples of one or more suspicious subgraph pattern(s), detect unseen nodes that occur in such pattern(s)*. While a GNN can be extended to enable pattern finding of suspicious motifs, weighted and temporal properties are often not explicitly incorporated. As a result, the dynamics of the network are not captured by the model.

1.2 Our Contributions

In this research, we build upon the work of Egressy et al. (2024) by enhancing and proposing techniques for graph neural networks to leverage the underlying network structure of transaction data for detecting suspicious accounts indicative of money laundering. Instead of directly extracting suspicious subgraph patterns from the network, the model performs supervised node classification to identify accounts that show structural resemblance to nodes in a target pattern.

In addition to utilizing ego IDs, binary node features designed to detect cycles, we propose a novel message passing scheme termed signed message passing, an alternative to existing heterogeneous schemes. While heterogeneous message passing aggregates messages from incoming and outgoing neighbours separately, signed message passing allows for the aggregation of both types of edges simultaneously while still preserving an indication of the directionality of edges.

Furthermore, we extend the money laundering patterns (ML patterns) to the context of weighted temporal networks by introducing the concepts of weighted patterns and temporal patterns. By incorporating edge weight multiplication with signed message passing, our model can detect patterns whose edge weights exceed a certain threshold. To identify temporally ordered patterns, we propose an LSTM aggregator that learns from the sequential order of edges imposed by timestamps. This LSTM aggregator is seamlessly integrated into the GNN layers, providing a straightforward and intuitive method of capturing the dynamic characteristics of financial networks.

The developed model, named LAS-GNN, is based on an embedding generation framework that can be used in an inductive setting, allowing for generalization across distinct networks. We test our algorithm on synthetic networks generated by ourselves and compare its detection performance against different baselines, including a graph isomorphism network benchmark that incorporates weights and timestamps as edge features. Although our findings can be generalized

and applied to other pattern detection tasks in dynamic network settings, such as molecule detection and traffic forecasting, our focus remains on the financial crime domain of money laundering.

This thesis is structured as follows. We begin with an overview of the related literature regarding approaches for money laundering detection in [Chapter 2](#). Scientific findings on standard machine learning algorithms and advanced techniques for incorporating network structure are discussed within the domain of AML. A more in-depth description of graph neural networks can be found in [Chapter 3](#), where we also describe current techniques that extend GNNs for pattern finding. In [Chapter 4](#), we introduce our proposed techniques and modifications to existing methods, offering a full description of our model. This is followed by [Chapter 5](#), which details our experimental setup, including the generation of synthetic networks, training specifications, compared models, and relevant evaluation metrics. The results of our model on the designed tasks are presented in [Chapter 6](#), where we also compare its performance to a graph isomorphism network and other baselines. The discussion and conclusion of our research can be found in [Chapter 7](#) and [Chapter 8](#), respectively.

Chapter 2

The State of Money Laundering Detection

In this chapter, we discuss recent scientific developments in Money Laundering Detection (MLD). There is a wide variety of models, which range from domain-led to entirely data-driven. The papers discussed here consist of standard, frequently used approaches as well as state-of-the-art models that together make up an overview of both effective and widely-used approaches for MLD. We refer to more extensive surveys for more information ([Labib et al.](#); [Chen et al., 2018](#)).

A problem related to money laundering detection is fraud detection, which shares many similarities but differs in *modus operandi* and patterns identified. While our main focus lies on money laundering detection specifically, we come across some results for fraud detection in the discussed literature. Two surveys on fraud detection specifically can be found here: ([Adewumi and Akinyelu, 2017](#); [Ali et al., 2022](#)).

Our overview of detection approaches for money laundering begins in [Section 2.1](#) with more traditional supervised machine learning and anomaly detection algorithms that directly operate on transaction data. Subsequently, we discuss graph-based approaches that explicitly utilize the graph structure of a financial network in [Section 2.2](#).

2.1 Transaction-Based Machine Learning for AML

In [Chapter 1](#), we discussed the downsides of the conventional rule- and scenario-based systems that are still primarily used at financial institutions today. Over the past decade, extensive research has been conducted on automated AI-based solutions aimed at money laundering detection, with more and more institutions embracing such methods alongside their traditional systems. These machine learning methods work on data and can be broadly categorized into two types: supervised and unsupervised. Although the former uses labelled data providing information about alerted cases, the latter discovers hidden patterns solely from unlabeled data. Both methods can be applied to transaction data, customer account data, or a combination of both. While the actual information available may vary across data sets or institutions, examples of features in transaction data consists of a transaction volume, date and time, currency, payment type, etc. Account data may comprise the type of account (person, corporate), affiliated bank, demographic information, and more. Note that the account features can be integrated into transaction features by creating corresponding feature columns for the source and target account of a transaction separately. Conversely, transaction features related to the same account can be combined (averaged, summed, etc.) to form account features, although such operations may affect the expressiveness of the original transaction feature values. Therefore, most studies focus on approaches analyzing transaction data.

A problem with transaction data is that the proportion of transactions related to financial crime is exceedingly small. While great for society, this can provide a challenge for learning algorithms. This phenomenon, where instances of a certain class are significantly underrepresented in the dataset, is referred to as *class imbalance*. Especially in domains such as financial crime,

where detecting the minority class is most crucial, it is important to take this into account when constructing and evaluating a machine learning model. When assessing a machine learning model using ground truth, certain performance measures, such as accuracy, can be misleading. A classifier that only predicts the majority class is bound to yield high accuracy, although such a model is essentially useless for MLD. Therefore, we mostly concern ourselves with the precision and recall scores. In our discussion of related literature, we focus on aspects such as the number of false positives, false negatives, etc. These metrics are explained in [Section 5.4](#).

2.1.1 Supervised Learning

In the case that labelled data of actual reported money laundering cases is available, it seems beneficial to leverage this information for creating a baseline understanding for the machine learning model.

In an empirical study conducted by [Zhang and Trubey \(2019\)](#), five prominent supervised machine learning algorithms are compared on the binary classification task of whether a transaction qualifies as a money laundering event or not. The authors used real-world transaction data provided by a non-disclosed U.S. financial institution. All transactions were initially alerted as suspicious, but less than one per cent was ultimately flagged as SAR. The objective was to distinguish the escalated alerts that were reported to the Financial Crimes Enforcement Network (FinCEN) from the inconsequential alerts. To handle the class imbalance, the authors also applied two sampling schemes to increase the relative presence of these events. These sampling schemes involved over-sampling the minority class (escalated alerts) and under-sampling the majority class (insignificant alerts). The implemented algorithms include Bayes logistic regression (BLR), decision tree (DT), random forest (RF), support vector machine (SVM), and artificial neural network (ANN). These models were built using ten non-disclosed explanatory variables from a set of roughly a hundred variables based on rich information from the transactions and customer characteristics, including risk rate, age and location.

Results revealed that without sampling, the ANN performed best with an Area Under the ROC Curve (AUC) of 0.74, while the SVM was the worst with an AUC of 0.60. However, while the ANN model showed resilience against the sampling schemes, the SVM became the second-best classifier in the case of under-sampling and even surpassed ANN as the best-performing model in the case of severe over-sampling. The finding that SVM performs well on more balanced data sets but not on the original, skewed data is reiterated by [Alarab et al. \(2020\)](#). The decision tree model gained from both sampling schemes but was consistently ranked as the worst performing classifier for each event rate, apart from the scenario without any sampling. Despite its lower predictive performance, decision trees can be valuable for identifying new rules that can conveniently be integrated into traditional rule-based systems, something done by [Wang and Yang \(2007\)](#) based on customer profiles of a Chinese bank. The random forest showed strong performance both with and without sampling.

[Alarab et al. \(2020\)](#) combined different supervised classification models, including RF, Extra Trees and Bagging classifier, into an ensemble model that outperformed each individual model in detecting illicit transactions in the Elliptic dataset, a dataset comprising real-world bitcoin transactions. This is accomplished by simply averaging the classification probabilities obtained by these methods. The ensemble method proved to reduce the number of false positives without increasing the false negatives. The authors also noted that one particular model, k-Nearest Neighbors (kNN), did not perform well and appears unsuitable for the task of money laundering detection. kNN relies on the k closest neighbours in the feature space to vote for the best class. However, due to the imbalanced dataset, the voting mechanism is highly likely to be skewed towards the majority class since the numerous negative instances (licit transactions) can easily outweigh the small number of positive instances (illicit transactions) in the neighbourhood of

a suspicious transaction. Furthermore, given that kNN is based on Euclidean distances, it is computationally expensive, especially in the case of high-dimensional data.

At last, we also want to mention the work of [Jullum et al. \(2020\)](#), whose supervised XGBoost model was trained on three types of historical data from Norway's largest bank: 1) normal, legal transactions, 2) transactions flagged as suspicious by the bank's internal alert system, and 3) potential money laundering cases reported to the authorities. Their work demonstrates that the common approach of not using normal or non-reported transactions in the training of a model can lead to sub-optimal results. Using a newly proposed performance measure specifically tailored for comparing their method with the bank's existing AML system, they show that their supervised model can outperform the traditional rule-based systems many banks rely on.

2.1.2 Unsupervised Learning

While labelled data can be of great value for the learning algorithm, it may not always be available. Certain financial institutions may lack access to labelled data sets due to privacy concerns, and manually preparing and labelling large data sets can be an expensive and time-consuming endeavour. Additionally, the labelled data may not truly match with the ground truth in the first place since 1) it is hard to know whether some alert is actually involved in money laundering and not just suspicious, and 2) a high percentage of money laundering cases are missed even by experts. Hence, it is unsurprising that extensive research in money laundering detection is concerned with unsupervised learning, where the machine learning techniques rely on a model without prior knowledge of the data or human supervision. The objective of such methods is to identify hidden structures and patterns as well as (dis)similarities among the data records.

The task of discovering rare occurrences in data is commonly known as anomaly detection. In the context of money laundering, this involves identifying unusual transactions or accounts whose behaviour deviates from that of typical accounts. A popular machine learning method employed for outlier detection is the isolation forest. Similar to a random forest, it consists of an ensemble of decision trees. However, rather than splitting nodes based on information gain derived from class labels, the trees in the isolation forest randomly split data points along a feature dimension and thus do not rely on any supervision. Isolation forests are successfully used as the main module for illicit transaction detection by [Shokry et al. \(2020\)](#) and as a subroutine within a broader detection framework by [Labanca et al. \(2022\)](#) and [Dumitrescu et al. \(2022\)](#). The linear time complexity and low memory usage renders them highly suitable for large datasets.

An alternative approach for outlier detection is the One-Class Support Vector Machine (OC-SVM). In contrast to the traditional SVM algorithm that separates the data into several classes, the OC-SVM model is an extension made by [Schölkopf et al. \(2001\)](#) that only considers a single "normal" class. To detect outliers, OC-SVM creates a boundary around the normal observations to isolate them from the anomalous ones. While [Alarab et al. \(2020\)](#) showed that traditional SVM performs poorly when the transaction data is highly imbalanced, it is precisely under these circumstances that OC-SVM shows great promise. [Tang and Yin \(2005\)](#) modified the OC-SVM algorithm for AML using a transaction database from an agriculture bank in China. The authors conclude that their method is simple and rapid and can be used in real-world systems without much modification. However, they acknowledge that the model parameters were determined through trial and error and could be tuned differently for better performance. In the absence of labelled data, model validation and evaluation can be challenging for unsupervised models.

Contrary to previous research, [Lorenz \(2021\)](#) suggest that unsupervised methods solely relying on anomaly detection are ineffective for real-life AML. They note that in some past experiments, data is synthetically generated to contain outliers by design, something that does not materialize in real-world data. By applying Uniform Manifold Approximation and Projection (UMAP) ([McInnes et al., 2018](#)) on Bitcoin transaction data to visualize the classifications of an isolation forest and the ground truth, the authors show that the illicit transactions in a real-life Bitcoin transaction

network are not outlying at all. The observation that not all outliers are illicit and not all illicit transactions are outliers is reasonable in AML, as sophisticated criminals generally attempt to mimic normal behaviour. This problem was previously acknowledged by Das et al. (2016) and is said to be a root cause for the high false positive and high false negative rates frequently associated with these methods. In both studies, the authors overcome this by applying anomaly detection algorithms in a broader *active learning* framework in which expert feedback is incorporated to adjust the detector during an interactive loop. These iterative "analyst-in-the-loop" approaches can be classified as *semi-supervised* since they learn patterns from mostly unlabelled data using small inputs from human supervision.

2.2 Graph-based Approaches for AML

The methods discussed in Section 2.1 learn directly from the observations specified in the data and do not explicitly leverage the financial network. In this section, we discuss models and approaches that do work with the relations specified by the underlying graph structure. After constructing a financial graph, with bank accounts and transactions represented by nodes and edges, respectively, as illustrated in Figure 2.1, one could apply one of many graph-based approaches. These models can be domain-led or data-driven, and they may explicitly search for suspicious money laundering subgraph patterns or not. In this section, we first discuss some domain-led approaches. Then, we proceed with graph-based machine learning models and discuss three promising techniques as seen in literature: directly using features derived from the network, generating graph embeddings using random walks, and utilizing more advanced deep learning methods. The latter will be the main focus of our research.

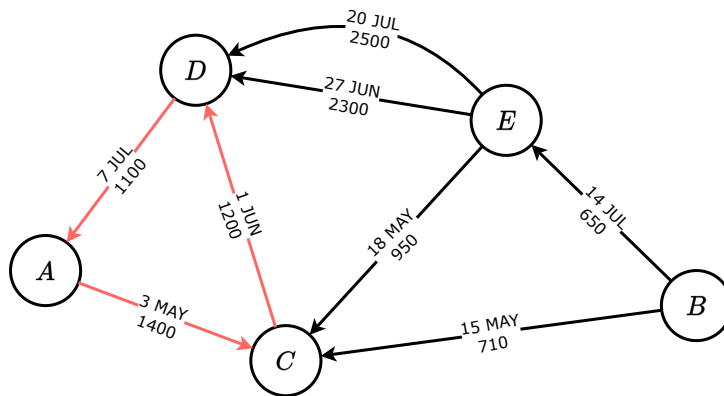


Figure 2.1: An example of a financial graph, where the edges include a date and an amount of money. Alerted transactions that together form a cycle are highlighted in red.

2.2.1 Domain-led

While the primary focus of this thesis is on learning algorithms, we want to mention some notable algorithms for MLD that do not exactly *learn* in a data-driven manner but use the graph data in a direct and algorithmic way. These approaches are more domain-led and depend on similarity measures, database processing, and threshold modelling.

Li et al. (2020) note a shortcoming of several general-purpose anomaly detection techniques, such as Fraudar (Hooi et al., 2016), D-Cube (Shin et al., 2017), and HoloScope (Liu et al., 2018), that have been developed for discovering structural anomalies in financial graphs. These approaches do not focus on money flow across multiple nodes, which is essential for ensuring accuracy and resilience against camouflage in ML activities. Instead, the authors propose a novel *anomalousness metric* that captures two money laundering specific traits, namely that 1) transfers

are dense due to high volume funds that flow between few nodes, and 2) intermediate accounts serve as a bridge and therefore often have a zero-out balance between their weighted in-degree and out-degree. Their proposed FlowScope algorithm extracts the densest tripartite subgraph from the network according to this metric in near-linear time using a priority tree. In order to find more suspicious groups of nodes, the densest subgraph is removed from the network, and the algorithm gets rerun. FlowScope is primarily tested on tripartite subgraphs with a single middle layer, although it also outperforms the previously mentioned algorithms in scenarios involving two layers of intermediate nodes.

Starnini et al. (2021) focus on identifying two specific smurf-like motifs in time-evolving transaction networks, namely the gather-scatter and scatter-gather patterns. They detect such patterns rapidly, using standard database joins that find common neighbours between different pairs of nodes within the same time window. However, their approach is predominantly reliant on a series of pre-processing and post-processing steps. Initially, trustworthy accounts or accounts with activity above a certain threshold are removed, as well as transactions involving a low amount of money. Furthermore, they ensure that each path of length two includes at least one cross-border transaction. They use a specific threshold for the maximum inter-transaction time window, along with constraints for the minimum total money flow and flow ratio for filtering the suspicious subgraphs in post-processing. The authors extract some notable patterns, visualized using flowcharts. Since this approach relies heavily on rules and thresholds, they intentionally leave the values non-disclosed.

Similar thresholds are also used in the research of Soltani et al. (2016). They incorporate the temporal and amount similarity between two transactions using a single equation, which is used to match transactions where the receiver account of one transaction corresponds to the sender account of the other. The authors subsequently construct a new graph exclusively containing these matching transactions and proceed to filter the nodes based on a designated *balance score*. Nodes with a balance score exceeding a certain threshold are considered potential money laundering accounts and are subjected to further analysis using a *structural similarity* metric. Finally, groups of money launderers are found by an algorithm that iteratively merges nodes that have at least one common neighbour. Experiments conducted on synthetically generated data show that their framework can effectively find scatter-gather patterns without specifically injecting this type of pattern.

The modelling of relationships in constructed graphs has been extensively researched in Social Network Analysis, and insight from this domain can also be applied in the context of money laundering detection. Dreżewski et al. (2015) construct social networks from bank statements and the National Court Register and assign roles to nodes based on node centralities such as closeness and betweenness, page rank, authoritativeness, and other measures. The purpose of these metrics is to determine the associations between accounts and the strength of their connections depending on their financial actions. The authors manage to point out leaders and vulnerabilities in the network, paving the way for more in-depth investigations into their connections. A similar approach is undertaken by Colladon and Remondi (2017) and they advocate for combining the centrality measures with machine learning approaches for detecting suspicious accounts.

2.2.2 Graph-based Features

Note that the domain-led approaches discussed in the previous section do not learn from the data and are hence limited to the static implementation of the algorithms. Consequently, they cannot adapt to new complex money laundering patterns and rich feature information cannot be incorporated. To overcome this, we now return to the machine learning algorithms described in Section 2.1. However, instead of directly fitting some classification model to the transaction data, there has been a lot of interest recently for incorporating the network structure in these learning

algorithms. While the research on graph-based machine learning models for MLD is increasing, it has only been prominent in the last three years (Bakhshinejad, 2023).

The most obvious way to incorporate graph structure into a machine learning model is by extending it with features derived from the network. The benefit of specifically designed features is that they can be based on insights into money laundering motifs and help the machine learning algorithm's findings become more easily accepted by banking experts. Note that they can help increase the performance of both supervised and unsupervised models. Straightforward examples of such features are the incoming- and outgoing degree of a node and node centrality measures, such as the ones mentioned in Section 2.2.1.

Dumitrescu et al. (2022) expand upon previous research on *egonet* features for fraud detection, as initially explored by Akoglu et al. (2010). The *egonet* of a specific center node v is the induced subgraph of v and all its immediate neighbours, including the ties among the neighbours. The authors propose a similar subgraph, termed the *reduced egonet*, where the nodes connected to v with only a single edge are removed. It turns out that some features of the reduced *egonet* are biased towards the extreme for certain nodes in ML patterns compared to normal nodes. Metrics such as the (relative) out-degree of the reduced *egonet* are clearly informative for anomaly detection and can be used as indicators for money laundering. Additionally, the authors compute random walks on the input graph to extract information on the amount of money circulated over long paths, especially in the case of cycles. For each node k , they generate random walks with given maximum length r , starting or ending in k , and they keep track of the minimum amount transferred along the random walk. The assumption the authors make is that in the usual case there is little correlation between the transactions of the nodes along the random walk, and so this amount is expected to be small. When the amount is large, it may signify money deliberately circulated through many accounts in an attempt to cover their origin. The maximum of this minimum value over all random walks of a node k of length at most r is used together with the features derived from the reduced *egonet* as the inputs for an isolation forest to detect accounts in ML patterns such as fan-in and gather-scatter. Apart from using random walks to calculate direct features, they can also be utilized for generating node embeddings, discussed in Section 2.2.3.

The streaming model proposed by Sun et al. (2022) introduces the MonLAD algorithm, which sequentially processes edges based on their corresponding transaction timestamps. At each step, statistical features of the nodes affected by that transaction are updated. By keeping track of the account residual and balance scores (relating to the amount of money residing in the account), the authors model a key trait of typical money laundering accounts, namely *frequently balancing*: accounts crave to frequently reach a balanced state by fan-outs upon receiving all the money of multiple fan-ins. The additional traits of *fast-in and fast-out* and *high throughput* are modelled by analyzing the frequency of balancing and the magnitude of imbalances observed. The concepts of balancing statistics are similar to the proposed FlowScope (Li et al., 2020) and structural similarity (Soltani et al., 2016) models discussed in Section 2.2.1 and thus the MonLAD algorithm can be considered domain-led. However, in contrary to those algorithms, the authors use these statistics as input features for their proposed anomaly detection algorithm AnoScore, that identifies outliers based on statistical deviation from a generalized Pareto distribution. Experiments show that MonLAD surpasses baselines such as FlowScope in detecting injected ML anomalies while also scaling linearly with the number of edges of the stream.

An alternative method to incorporate the network into a set of features is by first extracting small communities of related accounts centred around a single node. Subsequently, existing features, such as demographic and transaction features, of the parties within a specific community are aggregated to form community-broad features. This approach is adopted by Savage et al. (2016), and the resulting community features, along with dynamic features derived from time-series analysis (burst analysis providing an indication of transaction regularity), serve as input for supervised learning using SVM and random forest.

2.2.3 Random Walk Embeddings

While graph-based features for a node can be computed directly from the network, an alternative approach is to learn node representations. Node representations, also referred to as node embeddings, are low-dimensional vector representations of nodes that capture the structural and relational information such that it preserves the graph's topology. A classical example of a node representation learning algorithm is DeepWalk (Perozzi et al., 2014), whose generated embeddings are used as input for downstream machine learning tasks for AML by Wagner (2019). Based on techniques from the field of Natural Language Processing, DeepWalk performs a series of uniform random walks in the input graph. Such a random walk is treated as a "sentence", where the visited nodes are the "words". Truncated random walks are fed into the SkipGram algorithm that learns the probability distribution of the co-occurrence of nodes, from which the node representations can be derived.

The Node2Vec algorithm (Grover and Leskovec, 2016) improves upon DeepWalk by introducing the concept of biased random walks, where the walks are guided by two parameters that control the level of local (BFS) and global (DFS) exploration. In the work of Yuan et al. (2020), both DeepWalk and Node2Vec embeddings are fed into an OC-SVM and compared in the context of fraud detection in Ethereum transaction networks. Their results in detecting phishing scams show that the model using the node representations learned by Node2Vec yields better precision and recall. The effectiveness of Node2Vec is also investigated specifically for money laundering detection by Bakhshinejad (2023). Using 100,000 transaction records of the synthetic PAYSIM dataset (Lopez-Rojas et al., 2016), the authors carry out cross-validation experiments. They systematically vary parameters such as the size of the embedding vectors, the number of walks, and walk length to determine their impact on performance. Their findings indicate that increasing the size of the embedding vectors improves performance initially, although beyond a certain value, it does not significantly affect accuracy and may even reduce results marginally. Nevertheless, increasing the number of random walks and the length of the walks can shrink the false positive and false negative rates, as can be expected. However, it is important to keep in mind that increasing these parameters comes at the cost of significantly longer running times. Consequently, concerns arise regarding the feasibility of this approach on much larger networks, prompting a trade-off between computational resources and detection accuracy.

2.2.4 Deep Learning

The random walk embedding methods described in the previous section are based on the idea that nodes that are close in the graph have similar representations. While there are alternatives to purely proximity-based random walk embedding methods, such as the structural-based Struc2Vec (Ribeiro et al., 2017), these methods do not take any actual properties of the nodes into account. In other words, rich feature information from account or transaction data is completely ignored when the classification model solely relies on the embeddings generated by Node2Vec, for instance. Naturally, one can include these other features alongside the embeddings to form a larger feature set before applying any machine learning algorithm. However, it seems more beneficial to let the embeddings be guided by the features in the first place.

Fortunately, such methods exist. Graph Neural Networks (GNN) can learn node and graph embeddings by taking both the graph structure and the node features as input. It works by propagating information along the edges of the graph to update the node representations. This message passing, in combination with multiple layers and non-linear activation functions, can capture relationships and complex patterns in the data. In the last few years, GNNs have gained extreme popularity and different types of architecture variations are proposed and applied in many domains, including money laundering detection. The most widely used GNN model is the Graph Convolutional Network (GCN) introduced by Kipf and Welling (2017), described in more detail in Chapter 3.

The potential of GNNs for anti-money laundering was first demonstrated by [Weber et al. \(2018\)](#). The authors argue in favour of more advanced forensic analysis tools and state that deep learning for massive, dynamic graph data is more promising than the use of suspiciousness heuristics and must be considered an essential addition to current graph analytic approaches. They perform preliminary testing of graph neural networks applied to a large financial network (1M nodes, 9M edges), which is synthetically generated and injected with the basic ML patterns using the AMLSim data simulator ([Suzumura and Kanezashi, 2021](#)). Unfortunately, they do not provide classification results but solely report on the running times and scalability of GCN and FastGCN, which uses layer-wise sampling for efficiency. They are confident that with the development of high-performance code and memory-efficient graph representations, GNNs are well suited for handling dynamic transaction data at great scale.

In recent follow-up research, [Egressy et al. \(2024\)](#) apply all kinds of GNN variations to synthetic data generated by their updated data simulator AMLWorld ([Altman et al., 2024](#)). The models include additional techniques for the Graph Isomorphism Network, which is theoretically found to be the most powerful GNN ([Xu et al., 2019](#)), and the Graph Attention Network, which leverages attention mechanisms. The models and their modifications show great results in detecting the key ML patterns on both synthetic AML data and real-world Ethereum phishing data. The techniques of ego IDs (marking a center node with a distinct binary feature), port-numbering (for distinguishing multi-edges), and heterogeneous message passing (for indicating edge directionality), seem to broaden the expressiveness of the GNN architecture. While these techniques were proposed in earlier work of others separately, the authors prove both theoretically and experimentally that it is the combination of the three that enables the detection of ML patterns in directed multi-graphs. These techniques are described in more detail in [Chapter 3](#).

The mentioned models were all applied to large static graphs that contained all the transactions within a certain period. A common approach here is to use a sliding time-window to divide the network into subgraphs containing the edges whose timestamp are within a certain interval. [Weber et al. \(2019\)](#) experimented with integrating the temporal aspects of the data into the GNN and proposed EvolveGCN: a model that consists of multiple GCNs, namely one for each time-step, which are fed into a Recurrent Neural Network to capture the system dynamics. This way, the weights of the GCN in a future time-step are updated based on those from the past so that it captures the evolution of money laundering patterns. However, the model is highly complicated and has high computational complexity, making it unsuitable for large networks. Moreover, experiments with the Elliptic Dataset, using the 49 snap-shots in the dataset as time-steps, show that the improvement of EvolveGCN over standard GCN was not substantial at all. [Xia et al. \(2022\)](#) and [Alarab and Pragoonwit \(2023\)](#) also consider the combination of GCN with a recurrent neural network, namely Long Short Term Memory (LSTM). They propose using a GCN to explore spatial structure features in a financial network and then transferring the mined features to an LSTM for temporal feature extraction. Although temporal features can be integrated in this way, the network dynamics are not captured.

[Wan and Li \(2024\)](#) also note that existing literature is limited to representing node relationships in static graphs. Additionally, they state that performing graph convolutional operations for multiple time steps can cause information leakage issues and excessive smoothing. To address this, they propose the construction of dynamic graph snapshots with symmetrical spatiotemporal structures based on transaction information. In these graph snapshots, each transaction is treated as a node and currency flows are regarded as edge relationships. At all times, only one graph snapshot is retained, whose representation is updated after the transaction set of the current time step is processed, achieving dynamic, iterative updates for the model. At each time step, message passing in the graph snapshot is performed to extract spatial structural features, and the results are subsequently fed into an LSTM network to further extract the temporal features. The novelty lies in their approach to dynamic graph convolution, which combines the embedding values from the current graph convolution layer of the previous time step and the embedding

output from the previous layer at the current time step. Both embeddings are concatenated and fed into a multilayer perceptron (MLP) to update the node states at the current time step.

At last, we want to mention two other notable studies that utilize GNNs in their AML approach. Instead of using GNNs directly for classifying suspicious accounts or transactions, these methods perform anomaly detection in a more adversarial manner. [Li et al. \(2023\)](#) propose Diga, a guided diffusion model. Their semi-supervised model first employs a *subgraph sampler*, which extracts the most informative neighbours of each account based on biased PageRank and constructs a subgraph surrounding it. Secondly, these subgraphs are treated as independent training samples and are fed into a *guided diffusion model*. This diffusion model attempts to recover the input subgraph to an anomaly-free version. It first injects noise into the subgraph and subsequently applies GNN layers that try to reconstruct the original graph in a denoising process. Eventually, the reconstruction error is used as the anomaly score of each account.

The algorithm LaundroGraph by [Cardoso et al. \(2022\)](#) is completely self-supervised and thus does not use labelled data. Instead, it jointly trains an encoder and decoder that consist of multiple graph convolutional layers by sampling positive instances, which are directed edges in the graph, and non-edges as negative instances. Here, the existing edges are edges from an account node to a transaction node in a bipartite graph. For each instance, the K -hop subgraph also serves as input. The GNN is trained using a binary cross entropy loss function in a link prediction task between account-transaction pairs, which essentially corresponds to an anomaly detection task. The model learns the transaction behaviour of accounts, which serves as a reference point of expected behaviour used to score the anomalousness of new transactions entering the system. The novelty of this approach comes from the alternative graph representation used and the focus on applying the method to score newly incoming transactions, which is overlooked by most GNN approaches that classify nodes already present in the network.

Chapter 3

Graph Neural Networks

Although the Graph Neural Network (GNN) model was first proposed in 2005 by [Gori et al. \(2005\)](#) and subsequently generalized in 2008 by [Scarselli et al. \(2008\)](#), its application has only gained wide attention in recent years. In this chapter, we discuss two different settings for learning on graphs: transductive and inductive learning. We first discuss the most prominent example of the former and arguably the most influential GNN, the Graph Convolutional Network, in [Section 3.2](#). Then, we focus on a general inductive framework that seems more appropriate for our problem and describe the popular GraphSAGE and Graph Isomorphism Network models in [Section 3.3](#). Finally, in [Section 3.4](#), we discuss specific techniques that extend these inductive GNN models for pattern detection. For a more complete survey of GNN architectures, challenges, and applications, we refer to [Khemani et al. \(2024\)](#).

3.1 Transductive vs. Inductive

While graph neural networks are end-to-end frameworks useful in a variety of tasks, such as node classification and link prediction, they are essentially tools for generating embeddings similar to the random walk methods discussed in [Section 2.2.3](#). Such approaches often require that all nodes in the graph are present during training, meaning that they are inherently *transductive*. When training the model for node classification for instance, all information is used with the exception of the labels of the nodes in the test set. Consequently, transductive methods can only generate embeddings for a single fixed graph and they cannot learn to generalize across different graphs ([Song et al., 2022](#)). These methods are often useful in tasks where the network exhibits a certain degree of *homophily*, meaning that similar nodes are more likely to connect than dissimilar nodes. Consequently, nodes of the same type tend to cluster together, a phenomenon commonly observed in social network analysis ([McPherson et al., 2001](#)).

However, transductive learning seems less appropriate in evolving or dynamic graphs, since these methods are not able to efficiently generalize to unseen nodes; as soon as a new node is added, it is necessary to retrain the model. In these cases, an *inductive* framework seems better suited, where the model leverages node feature information to efficiently generate node embeddings for previously unseen data. We argue that for money laundering detection specifically this is more appropriate, since new transactions are made constantly and bank accounts can be opened and closed. Furthermore, it can be the case that labeled data (known money laundering cases) that we want to leverage comes from a different period or dataset than the data we want to make predictions on. Moreover, in our endeavour to identify money laundering patterns, we assume the network exhibits a certain degree of *heterophily*. Although suspicious nodes are connected to other suspicious nodes within the same ML pattern, such patterns tend to be relatively small. When we consider the suspicious nodes from distinct patterns, there is no assumption on how closely they are located in the graph. In other words, the ML patterns we seek can be distributed across the entire network and we must learn to recognize the structural properties of a node's neighbourhood that generalize between patterns. The inductive setting seems better suited to this notion of heterophily. Both settings are illustrated in [Figure 3.1](#).

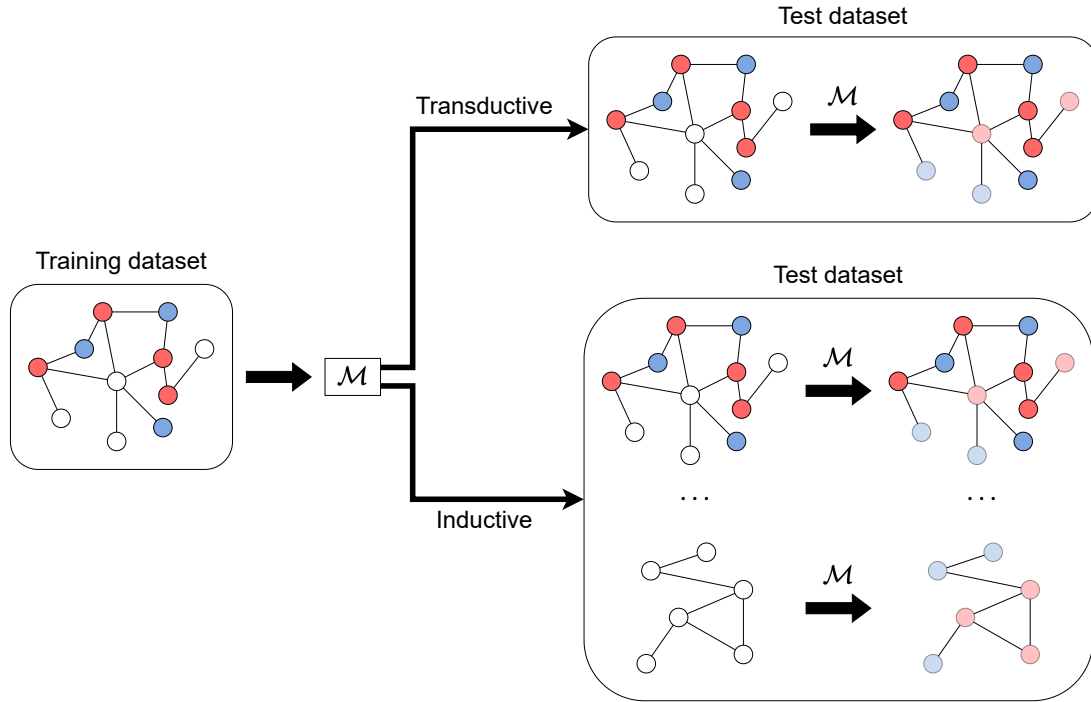


Figure 3.1: Transductive learning versus inductive learning for node classification. Image recreated from Song et al. (2022).

3.2 Graph Convolutional Network

The Graph Convolutional Network (GCN) is an inherently transductive method. Similar to most GNN variations that came after, GCN aims to generalize well-established neural models like Recurrent Neural Networks (RNN) and Convolutional Neural Networks (CNN) to work on arbitrarily structured graphs. They all share a somewhat general architecture where the objective is to learn a function on a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ which takes as input

- the adjacency matrix A , which describes the graph structure of $\mathcal{G}$,
- an $N \times D$ feature matrix X , which summarizes a feature description x_v for each node $v \in \mathcal{V}$ (N is the number of nodes, D the number of input features per node),

and produces a node-level output Z , which is an $N \times F$ matrix with F the number of output features per node. We can then write every neural network layer as a non-linear function f as follows. For any $k \geq 1$

$$H^{(k)} = f(H^{(k-1)}, A), \quad (3.1)$$

with $H^{(0)} = X$ and $H^{(K)} = Z$, where K is the total number of layers. The specific models then vary solely based on how $f(\cdot, \cdot)$ is chosen and parameterized.

A simplistic layer-wise propagation rule is

$$H^{(k)} = f(H^{(k-1)}, A) = \sigma(AH^{(k-1)}W^{(k)}). \quad (3.2)$$

Here $W^{(k)}$ is the weight matrix for layer k , and $\sigma(\cdot)$ is a non-linear activation function like ReLU. Although this propagation rule can already be quite powerful on its own, there are two limitations that need to be fixed.

First, in case there are no self-loops in the graph, the multiplication of A sums up all the feature vectors of the neighbouring nodes for each node, but excludes the node itself. A simple fix is to add the identity matrix I to A such that self-loops are enforced.

Secondly, A must be normalized such that all rows sum to one. Otherwise, the scale of the feature values will completely change after the multiplication. Normalization is often achieved by an additional multiplication with the inverse of the diagonal node degree matrix D . Multiplying with $D^{-1}A$ in the layer propagation rule now amounts to averaging neighbouring node features. Symmetric normalization, i.e. $D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$, is more commonly used and provides more interesting results.

After applying these two modifications, we essentially get the propagation rule used by the GCN model:

$$H^{(k)} = \sigma(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(k-1)}W^{(k)}). \quad (3.3)$$

where $\tilde{A} = A + I$ and $\tilde{D}$ is the diagonal node degree matrix of $\tilde{A}$.

In the training phase, the weights in the weight matrices $W^{(1)}, \dots, W^{(K)}$ are learned using a loss function. After forward propagation through the network, a sigmoid function is applied to the last layer, which yields the current predicted probability outcomes for the node class labels in case of node classification. For supervised learning, the cross entropy loss is then computed using the actual known class labels. For binary classification, this binary cross entropy loss function is as follows:

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^N -(y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)), \quad (3.4)$$

where p_i is the predicted probability of class 1 and y_i is the actual class label. The loss measures the difference between the predicted binary outcomes and actual binary labels, which is desired to be as low as possible. After calculating the loss, it is back-propagated and the weight matrices in each layer are updated using stochastic gradient descent.

GCN is considered a *spectral* GNN. In technical terms, the feature vectors of nodes are considered multi-channel graph signals, for which the model learns spectral filters in the Fourier domain defined by the eigenvectors of the graphs Laplacian matrix. This means that the filters learned are graph specific, which explains why GCN is transductive (Mishra et al., 2020).

3.3 Inductive Representation Learning

Spatial GNNs define the convolution operation directly on the structure of the graph. Instead of learning individual embeddings for each node in the graph, a spatial GNN learns a function that generates embeddings for unseen nodes by aggregating features from a node's surrounding neighbourhood. This way, it simultaneously learns the topological structure of such a neighbourhood as well as the distribution of the features of the nodes in it.

A powerful, spatial graph neural network framework that uses node feature information to inductively generate representations for unseen data is proposed by Hamilton et al. (2017). Pseudo-code of this embedding generation algorithm is shown in Algorithm 1. Provided as input are the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and a feature vector $\mathbf{x}_v$ for each node $v \in \mathcal{V}$, which initializes the embedding vectors $\mathbf{h}_v^{(0)}$ as can be seen in line 1. Let k denote the current step of the outer loop (line 2), which coincides with the current depth or layer within the GNN, and let $\mathbf{h}_v^{(k)}$ denote the embedding of node v at this step. In each iteration, we proceed as follows for each seed node $v \in \mathcal{V}$:

1. The representations of the nodes in the neighbourhood of v , $\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}(v)\}$, are aggregated by the AGGREGATE function into a single vector $\mathbf{a}_v^{(k)}$ (line 4). Note that this aggregation depends on the embeddings of the neighbouring nodes in the previous iteration, i.e. step $k - 1$.

Algorithm 1: Inductive embedding generation (Hamilton et al., 2017)

Input : Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; input features $\{\mathbf{x}_v, \forall v \in \mathcal{V}\}$; depth/number of GNN layers K ;
 aggregate and update functions $\text{AGGREGATE}^{(k)}, \text{UPDATE}^{(k)}, \forall k \in \{1, \dots, K\}$;
 neighbourhood function $\mathcal{N} : v \rightarrow 2^{\mathcal{V}}$

Output : Vector embeddings $\mathbf{z}_v$ for all $v \in \mathcal{V}$

```

1  $\mathbf{h}_v^{(0)} \leftarrow \mathbf{x}_v, \forall v \in \mathcal{V}$ 
2 for  $k = 1 \dots K$  do
3   for  $v \in \mathcal{V}$  do
4      $\mathbf{a}_v^{(k)} \leftarrow \text{AGGREGATE}^{(k)}(\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}(v)\})$ 
5      $\mathbf{h}_v^{(k)} \leftarrow \text{UPDATE}^{(k)}(\mathbf{h}_v^{(k-1)}, \mathbf{a}_v^{(k)})$ 
6   end
7 end
8  $\mathbf{z}_v \leftarrow \mathbf{h}_v^{(K)}, \forall v \in \mathcal{V}$ 

```

2. The node's current representation is updated by the UPDATE function (line 5), which takes as input the seed node's previous embedding $\mathbf{h}_v^{(k-1)}$ and the result of its neighbourhood aggregation $\mathbf{a}_v^{(k)}$.

After repeating this for each node for K iterations, the algorithm outputs the final node embeddings $\mathbf{h}_v^{(K)}$ (line 8). To optimize these output representations $\mathbf{z}_v$ for all $v \in \mathcal{V}$, we apply a loss function and adjust the weight matrices and parameters of the aggregator and update functions corresponding to each layer k , using stochastic gradient descent. This results in an end-to-end inductive algorithm suitable for node classification based on the learned node embeddings. An additional advantage of this inductive framework is its efficiency in terms of both memory and time. By conducting *mini-batch training*, we only consider the embedding generation for a small group of seed nodes (i.e. a batch) at a time, avoiding the need to process an entire, potentially massive, network at once—something that can become infeasible in transductive learning (Zhou et al., 2021). For each batch, a small number of nodes are randomly sampled, their neighbourhoods are extracted in a breadth-first manner using the neighbourhood function $\mathcal{N}$, and message passing is conducted in the corresponding subgraphs. Consequently, the weight matrices are updated more frequently (after processing each batch rather than the entire dataset), leading to faster convergence. Utilizing GPUs for processing different batches in parallel further enhances the training speed.

The two general AGGREGATE and UPDATE steps described above form the foundation of so-called Message Passing Neural Networks (MPNN), the most prominent type of spatial GNNs. These functions are to be determined and may vary for each layer k . By inserting different AGGREGATE and UPDATE mechanisms, this general framework for inductive representation learning can be adapted to accommodate different types of MPNNs, such as GraphSAGE, Graph Isomorphism Network (GIN), and Graph Attention Network (GAT). We discuss the former two in the next sections.

3.3.1 GraphSAGE

The general inductive representation learning algorithm was originally proposed for the *GraphSAGE* model (Hamilton et al., 2017), whose goal is to learn a function that generates embeddings by sampling and aggregating (SAmple and aggreGatE) features from a node's local neighbourhood. The neighbourhood function $\mathcal{N}$ uniformly samples a fixed-size set of neighbours at each depth k , which implies that not every neighbour contributes to the embedding of a node. The idea is that informative embeddings can be derived from only a subset of neighbours, making the

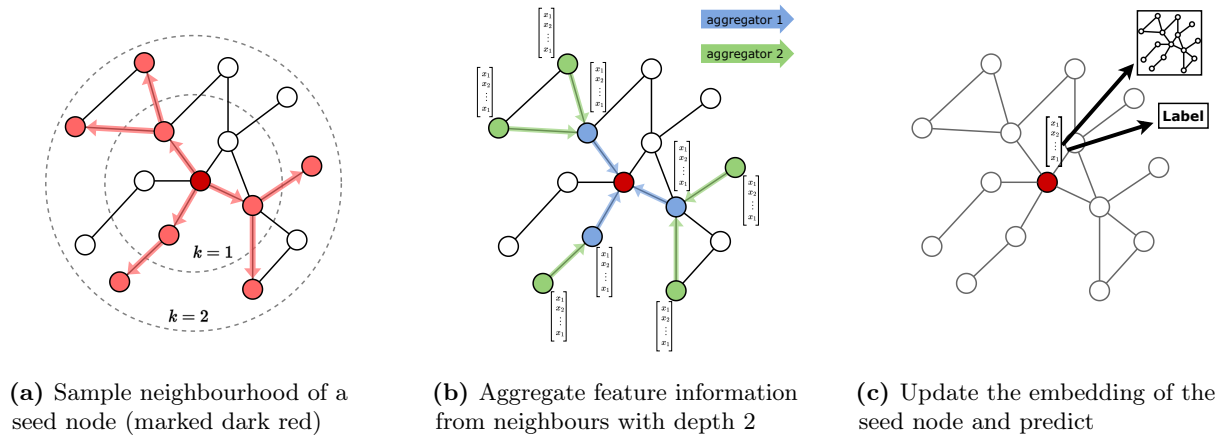


Figure 3.2: The sample and aggregate approach of GraphSAGE (Hamilton et al., 2017)

process more time-efficient. However, a clear drawback of such a sampling-based design is that a lot of neighbourhood information is lost. When complete structural information is essential, as in our pattern detection tasks, it is essential to utilize the full neighbourhood. The sample and aggregate scheme of GraphSAGE is illustrated in Figure 3.2.

The UPDATE step used in GraphSAGE is as follows:

$$\mathbf{h}_v^{(k)} \leftarrow \sigma \left(\mathbf{W}^{(k)} \cdot \text{CONCAT}(\mathbf{h}_v^{(k-1)}, \mathbf{a}_v^{(k)}) \right).$$

We can see that the previous embedding vector is concatenated with the aggregated neighbourhood vector, which is subsequently fed through a fully connected layer with nonlinear activation function σ . Note that $\mathbf{W}^{(k)}$ is the weight matrix for layer k to be optimized during stochastic gradient descent. For the AGGREGATE step, the authors suggest three types of aggregators: mean, max-pooling, and LSTM. The mean aggregator straightforwardly computes the element-wise mean of the embedding vectors in the neighbourhood, which is nearly equivalent to the convolutional propagation rule used in the GCN framework; when we skip the concatenation in the UPDATE rule, we essentially get an inductive variant of the GCN model! The max-pooling aggregator feeds the neighbours' embeddings through a fully connected neural network and takes the element-wise maximum. Finally, the LSTM aggregator, as the name suggests, utilizes a Long Short Term Memory network within the aggregation mechanism to further enhance the expressive capabilities of the model. Since LSTMs are not inherently symmetric (i.e., they are not permutation invariant) due to processing inputs as a sequence, the authors adapt the LSTMs to operate on an unordered set by simply applying the LSTMs to a random permutation of the node's sampled neighbours. This approach disregards any sequential ordering that may persist in the data, which is correct for most applications that deal with static input graphs. However, for dynamic networks it seems beneficial to allow the LSTMs to learn the sequential order that persist between edges, which is what our proposed LSTM aggregator detailed in Section 4.4 aims to do.

In their experiments, the authors demonstrate that GraphSAGE is effective in a wide range of node classification tasks from multiple domains, using data of citation networks, Reddit posts, and protein interactions. They specifically test on such dynamic graphs, showcasing the inductive power of the architecture, yielding significantly better results than DeepWalk while being 10-100 times faster, depending on the neighbourhood sampling size. In the experiment with the protein interactions dataset, the authors explicitly test the generalization capabilities of the inductive learning framework across multiple graphs.

3.3.2 Graph Isomorphism Network

In their attempt to understand the representational properties and limitations of GNNs, [Xu et al. \(2019\)](#) developed a theoretical framework for analyzing the expressive power of GNNs in capturing different graph structures. They present a MPNN architecture that is provably the most expressive among the class of GNNs, namely the *Graph Isomorphism Network* (GIN). The GIN model generalizes the Weisfeiler-Lehman graph isomorphism test ([Lehman and Weisfeiler, 1968](#)), a heuristic test for the existence of an isomorphism between two graphs, thus achieving maximum discriminative power among GNNs. The authors utilize multi-layer perceptrons (MLPs) to model and learn the functions comprising the AGGREGATE and UPDATE steps in the message passing mechanism. Since a MLP can represent the composition of functions, the node-wise propagation rule can be written as a single step, as shown in [Equation 3.5](#).

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(k-1)} \right), \quad (3.5)$$

where ϵ is either a learnable parameter or a fixed scalar that essentially determines the importance of the previous embedding of seed node v . Note that at each depth k , and so for each GNN layer, there is a distinct MLP with its own set of learnable weights.

3.4 GNN Techniques for Graph Pattern Finding

In their paper *Provably Powerful Graph Neural Networks for Directed Multigraphs*, [Egressy et al. \(2024\)](#) propose a set of simple and intuitive techniques that can transform any MPNN into powerful directed multigraph neural networks with provable properties. They show both experimentally and theoretically that GNNs can identify any directed subgraph pattern when equipped with (1) ego IDs, (2) heterogeneous message passing, and (3) port numbering. The combination of these three techniques applied to suitably powerful standard MPNN yields so-called *universal* GNNs – those that can distinguish any two non-isomorphic (sub-)graphs, while not mistakenly distinguishing any two isomorphic (sub-)graphs.

We now explain these techniques. Since we will not be focussing on the multigraph setting (see [Section 5.1](#)), we leave out port numbering ([Sato et al., 2019](#)) from this discussion, whose intent is to distinguish parallel edges.

3.4.1 Modelling Node Identity

First, the authors propose to incorporate node identity into the message passing, which seems essential for detecting subgraph patterns that consist of (undirected) cycles. A natural approach would be to assign the unique node index $i \in \{1, \dots, n\}$ of each node as node ID and to use one-hot encoding vectors $\mathbf{v}_i$ of length n , with $\mathbf{v}_i[i] = 1$ and 0 elsewhere, as node input features. While this yields a *universal* GNN, it heavily relies on the initial ordering of the nodes, of which there are $n!$ possible permutations. Consequently, nodes with identical structural neighbourhoods fail to be mapped to the same embedding, resulting in low generalization capability ([Egressy and Wattenhofer, 2022](#)), rendering this approach unsuitable for the inductive setting. The same issue arises when assigning random features ([Abboud et al., 2020](#); [Sato et al., 2021](#)) as input node features.

However, functions that can assign node ID vectors inductively have been proposed. The ID-GNN-Fast method of [You et al. \(2021\)](#) injects node identity information via augmented node features using cycle counts, efficiently computable through powers of a graph’s adjacency matrix. A similar method is proposed by [Egressy and Wattenhofer \(2022\)](#), who precompute positional node embeddings. Here, the input graph is embedded into a Euclidean space approximating l_∞ and the coordinate locations are used as node features.

The more straightforward approach taken by [Egressy et al. \(2024\)](#) is to set the so-called *ego ID* feature of a specific seed node to be 1, while all other nodes in its neighbourhood are assigned an ego ID of 0. The idea is intuitive: by “marking” a “center” node with a distinct binary feature, this node can recognize when a sequence of messages cycles back around to it. As a result, the GNN can detect the cycles a node is part of. Note how the ego ID is a scalar that can be easily concatenated to already existing node features and whose value dynamically changes based on whether it is sampled as a seed node, meaning it is the sampled node in mini-batch training whose neighbourhood is extracted and for which we want to make a prediction. In practice, the implementation of ego IDs can be seen as a preprocessing step integrated with the creation of batches during mini-batch training, detailed in [Section 4.6](#).

3.4.2 Modelling Edge Directionality

Most traditional graph neural network applications consider the underlying structure of the input graph to be undirected. In that case, the generic aggregation and update mechanisms are as formulated in [Algorithm 1](#), where the neighbourhood function $\mathcal{N}(v)$ simply returns the set of all neighbours of v , which includes both nodes connected by an incoming edge and an outgoing edge, i.e. $\mathcal{N}(v) = \mathcal{N}_{\text{in}}(v) \cup \mathcal{N}_{\text{out}}(v)$. As a result, the message passing occurs in a bidirectional manner, termed *bidirectional message passing*: messages traverse in both directions over an edge, and both endpoints aggregate each other’s embeddings. However, a significant drawback of this approach is that it does not consider edge directionality, even though this can be crucial information. In the domain of financial networks and money laundering, differentiating between the sender and the receiver is paramount. The ML subgraph patterns we aim to identify are partly characterized by their suspicious combination of edge directions.

Alternatively, one could consider a strategy where the message passing strictly follows the direction of the edges, which we refer to as *directional message passing*. In this approach, a node sends its embedding solely along its outgoing edges while aggregating messages received exclusively through its incoming edges, as illustrated in [Equation 3.6](#).

$$\mathbf{a}_v^{(k)} = \text{AGGREGATE}^{(k)}\left(\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}_{\text{in}}(v)\}\right), \quad (3.6)$$

However, this method may lead to a loss of information since the embedding of nodes with zero in-degree cannot be computed (its initial embedding of the input features remains unchanged), rendering the GNN unable to differentiate such nodes. Moreover, nodes with zero out-degree cannot contribute to the embedding of any other node.

The aim is to conduct message passing in both directions while still indicating the edge direction in some way. [Egressy et al. \(2024\)](#), and simultaneously [Rossi et al. \(2024\)](#), propose using a separate message-passing layer for the incoming and outgoing edges, respectively. Since this is equivalent to using a relational GNN with two edge types ([Schlichtkrull et al., 2018](#)) and their implementation involves converting the underlying network to a heterogeneous graph, we refer to this as *heterogeneous message passing*. The aggregation and update mechanisms are shown in [Equation 3.7](#).

$$\mathbf{a}_{\text{in},v}^{(k)} = \text{AGGREGATE}_{\text{in}}^{(k)}\left(\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}_{\text{in}}(v)\}\right), \quad (3.7a)$$

$$\mathbf{a}_{\text{out},v}^{(k)} = \text{AGGREGATE}_{\text{out}}^{(k)}\left(\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}_{\text{out}}(v)\}\right), \quad (3.7b)$$

$$\mathbf{h}_v^{(k)} = \text{UPDATE}^{(k)}\left(\{\mathbf{h}_v^{(k-1)}, \mathbf{a}_{\text{in},v}^{(k)}, \mathbf{a}_{\text{out},v}^{(k)}\}\right), \quad (3.7c)$$

where $\mathbf{a}_{\text{in}}$ and $\mathbf{a}_{\text{out}}$ are the aggregations of incoming and outgoing neighbours, respectively. The update function now takes both these separate aggregations as inputs, along with its previous embedding, to compute the new embedding of a node. However, we denote one flaw of this

approach. By treating the aggregations of the different edge types separately, the aggregation function of one edge type lacks access to the embeddings of the other edge type. While this certainly helps the model to discern the edge directions in ML patterns, it sacrifices the flexibility to learn correlations between oppositely directed edges since the aggregation procedure cannot cope with both edge types simultaneously. The latter is especially crucial when we want to model time dependencies between incoming and outgoing transactions. To solve this issue, we introduce a novel message passing scheme that allows the simultaneous aggregation of both edges types while preserving directionality. We refer to this approach as *signed message passing* and it is described in [Section 4.2](#).

3.4.3 Modelling Weights

As is often the case in network domains, we can associate weights to the edges. Although we do not specify what these weights represent, they could reflect various factors in the context of a financial network, such as the pair-wise similarity between two nodes (for instance, based on account-based features, transaction behaviour, etc.), the transaction volume, the frequency of interactions, or a combination. A way of handling such weights is to include the weights as an edge feature co-embedded with the node features. [Hu et al. \(2020\)](#) extend the GIN model described in [Section 3.3.2](#) to incorporate edge features into the aggregation procedure as follows:

$$\mathbf{h}_v^{(k)} = \text{MLP}^{(k)}\left(\left(1 + \epsilon^{(k)}\right) \cdot \mathbf{h}_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} \text{ReLU}\left(\mathbf{h}_u^{(k-1)} + \mathbf{e}_{u,v}\right)\right). \quad (3.8)$$

Here, $\mathbf{e}_{u,v}$ is a vector of the edge features of edge (u, v) , possibly transformed by a linear layer to match the size of $\mathbf{h}_u^{(k-1)}$. While [Egressy et al. \(2024\)](#) do not model edge weights explicitly, they do utilize this extended GIN model to handle other kinds of edge features contained in their AMLWorld dataset.

3.4.4 Modelling Time

To the best of our knowledge, there is no previous work that aims to use GNNs to detect subgraph patterns that exhibit explicit temporal characteristics. Since an edge in the financial network represents a transaction, each edge can be associated with a specific timestamp indicating when the transaction occurred. While various methods for temporal modelling in AML, such as sliding time windows, have been discussed in [Chapter 2](#), a straightforward way for incorporating time is to include the timestamps as edge features in the model, similar to the approach for edge weights described previously. However, this only deals with time implicitly by making the temporal information available to the model. Instead, we propose to model the temporal dependencies between transactions explicitly by utilizing LSTMs in the aggregation step. Our approach, along with other modifications to these GNN techniques, are described in the next chapter.

Chapter 4

Method

In the previous chapter, we discussed related work on techniques that extend GNNs for pattern finding. Most notably, the work of [Egressy et al. \(2024\)](#), who propose the combination of ego IDs and heterogeneous message passing. In this chapter, we extend the pattern detection task to weighted and temporal settings, and propose additional modifications and techniques for coping with these characteristics. In [Section 4.1](#), we first define the subgraph patterns we aim to detect. We proceed by discussing our novel message passing scheme ([Section 4.2](#)), edge weight multiplication ([Section 4.3](#)), and the LSTM aggregator ([Section 4.4](#)). Finally, we present our model’s architecture in [Section 4.5](#) and describe some implementation details in [Section 4.6](#).

4.1 Money Laundering Patterns

The financial network is a *directed multigraph*, as can be seen in the example in [Figure 2.1](#). Given such a graph as input, the question becomes: what type of structures are we looking for that may imply money laundering? An example of such a topology is a simple cycle, which denotes a series of transactions in which the money ultimately returns to the bank account it was initially sent from. The presence of such cycles in the financial network is a strong indicator of potential money laundering. Similarly, money launderers can use various mixing and shuffling schemes, which can be expressed in terms of subgraph structures ([Granados and Vargas, 2022](#); [He et al., 2021](#); [Weber et al., 2018](#); [Starnini et al., 2021](#)). [Suzumura and Kanezashi \(2021\)](#) introduced a set of eight common money laundering patterns, or so-called motifs, illustrated in [Figure 4.1](#).

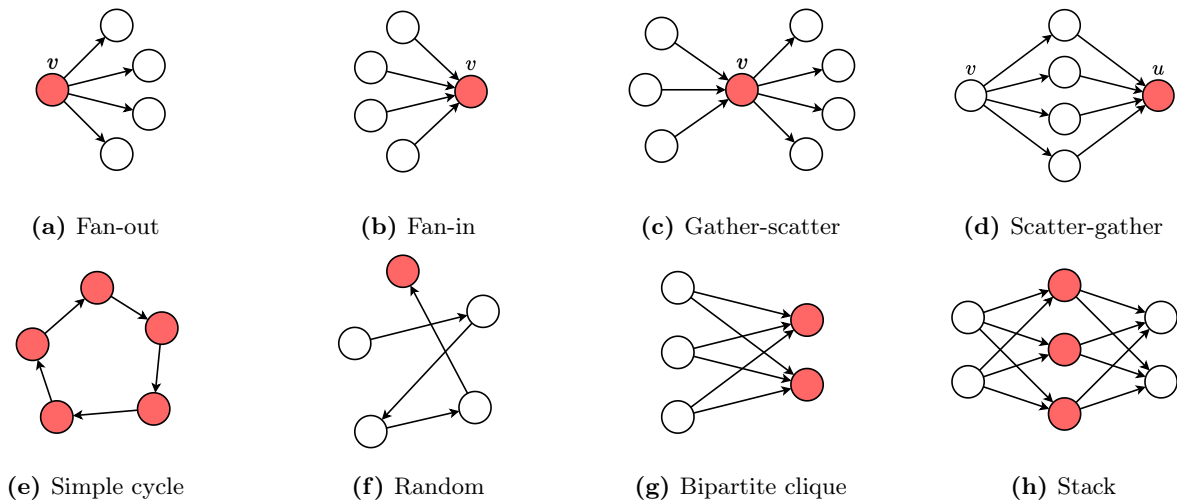


Figure 4.1: Eight key money laundering patterns as identified by [Suzumura and Kanezashi \(2021\)](#). The nodes marked in red can be seen as key accounts and are the nodes to be detected in a node classification task.

Since we perform node classification to identify nodes that are part of a ML pattern instead of fully extracting the patterns, we use the concept of a *key node* to designate a specific node by which a pattern is identified. To be more precise, we use the following definitions for when a pattern is considered suspicious and what its respective key node is:

- **Fan-out:** A fan-out pattern consists of a node with out-degree > 3 . Naturally, this source node is the key node we aim to detect.
- **Fan-in:** A fan-in pattern consists of a node with in-degree > 3 . This sink node is the key node.
- **Gather-scatter:** A gather-scatter pattern consists of a fan-in followed by a fan-out of the same node. Here, the intermediate node is the key node and it must have both in- and out-degree ≥ 3 .
- **Scatter-gather:** In case a fan-out and fan-in of two separate nodes are connected to each other by the same set of at least two intermediate nodes, we get a scatter-gather pattern. We designate the sink node to be the key node that needs to be identified.
- **Simple cycle:** For a simple cycle pattern, the sequence of edges starts and ends with the same node and visits each intermediate node only once. Since each node in a cycle is equivalent, they are all regarded as key nodes that need to be detected.
- **Bipartite clique:** A bipartite clique pattern consists of a set of at least two input nodes that are all connected to the same set of at least two output nodes. The sink nodes are considered key nodes.

In our experiments, we do not include the random and stack patterns. The key nodes of a pattern are marked red in [Figure 4.1](#). Note that the thresholds (such as the minimum degree or number of intermediate nodes) used to define whether a pattern is suspicious are arbitrary and can be changed. We selected these definitions and assignment of key nodes, although any other definition or assignment would also be effective as long as it is consistent.

Unfortunately, these patterns are rather generic and also common amongst perfectly innocent transactions. Therefore, we extend the definition of suspicious patterns to include weighted and temporal characteristics that are evident in dynamic networks. We introduce the notion of weighted patterns and temporal patterns by including edge weights and edge timestamps, respectively. As stated in [Chapter 3](#), edge weights can signify connection strength or pair-wise similarity between two nodes, while edge timestamps impose a temporal order. Generally, we say that a **weighted pattern** is a pattern constrained by the edge weights, and a **temporal pattern** is a pattern constrained by the sequential order of edges imposed by the edge timestamps. The specific definitions we use, along with examples, are given in [Section 5.1.3](#).

4.2 Modelling Edge Directionality: Signed Message Passing

In [Section 3.4.2](#), we discussed how [Egressy et al. \(2024\)](#) and [Rossi et al. \(2024\)](#) proposed heterogeneous message passing to account for edge directionality during the message passing. However, the limitation of this approach is that by using separate message passing layers for the incoming and outgoing edges, relations between the two edge types cannot be learned by the aggregator. Therefore, we propose to indicate edge direction by weighting the embeddings over the two directions differently within a single aggregation function, a method we call *signed message passing*. Instead of aggregating the embeddings over incoming and outgoing edges separately, the aggregation function receives both as input. However, each neighbour’s message is multiplied

by a directional vector $\mathbf{d}$, which is a linear embedding of 1 for incoming neighbours and -1 for outgoing neighbours, see [Equation 4.1](#).

$$\mathbf{a}_v^{(k)} = \text{AGGREGATE}^{(k)}\left(\{\mathbf{d}_{u,v} \odot \mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}(v)\}\right), \quad (4.1a)$$

where

$$\mathbf{d}_{u,v} = \begin{cases} \text{Linear}(1) & u \in \mathcal{N}_{\text{in}}(v), \\ \text{Linear}(-1) & u \in \mathcal{N}_{\text{out}}(v). \end{cases} \quad (4.1b)$$

Here, *Linear* is a linear layer that transforms an input scalar into a vector the size of the node embeddings, and $\odot$ represents element-wise multiplication. The latter is now possible because both factors have the same dimension due to the linear layer. Instead of simply multiplying every component of $\mathbf{h}_u^{(k-1)}$ with the scalar 1 or -1 , the model can learn a separate weight factor for each component. This allows the model to determine how and which parts of the embedding to differentiate for the two different directions. Note that we specifically rely on the sign to discern between the two directions and that the actual scalar value of 1 is arbitrary and can be changed to any value w , something we demonstrate in the next section when we introduce edge weights.

4.3 Modelling Weights: Edge Weight Multiplication

We include the possibility of edge weights in our model. A weight $w_{u,v} \in [0, 1]$ is assigned to an edge (u, v) and can be interpreted as the strength of a connection between the two nodes. The general idea is that we consider a ML pattern suspicious if the weights of the edges included in the pattern are all high (with respect to some threshold). For more detailed information on how we generate the weights, we refer to [Section 5.1.3](#).

Looking at [Section 4.2](#), the extension to include edge weights to signed message passing is quite straightforward. Instead of using the default weight of 1 for original edges and -1 for reverse edges, we can use w and $-w$, as follows:

$$\mathbf{d}_{u,v} = \begin{cases} \text{Linear}(w_{u,v}) & u \in \mathcal{N}_{\text{in}}(v), \\ \text{Linear}(-w_{v,u}) & u \in \mathcal{N}_{\text{out}}(v). \end{cases} \quad (4.2)$$

As a result, the message transferred over that edge is multiplied by the embedding of the corresponding weight. This seems natural since the weight then determines the importance of that message. We argue that the integration of edge weights and signed message passing is particularly intuitive in financial networks, where negative weights correspond to negative transactions in the opposite direction.

This edge weight multiplication is an alternative to treating the weights as an edge feature co-embedded with the node features, as described in [Section 3.4.3](#). In that case, the weight is also available to the model, but the model itself must learn how to utilize it effectively.

4.4 Modelling Time: LSTM Aggregation

In addition to edge weights, we explicitly model time dependencies and integrate temporal information from edge timestamps to identify ML patterns that are also suspicious with respect to this component. We first describe the workings of the LSTM, the primary component used to model sequential dependencies, and then explain how it integrates into our GNN as an aggregator. Details on generating the edge timestamps in our synthetic networks are provided in [Section 5.1.3](#).

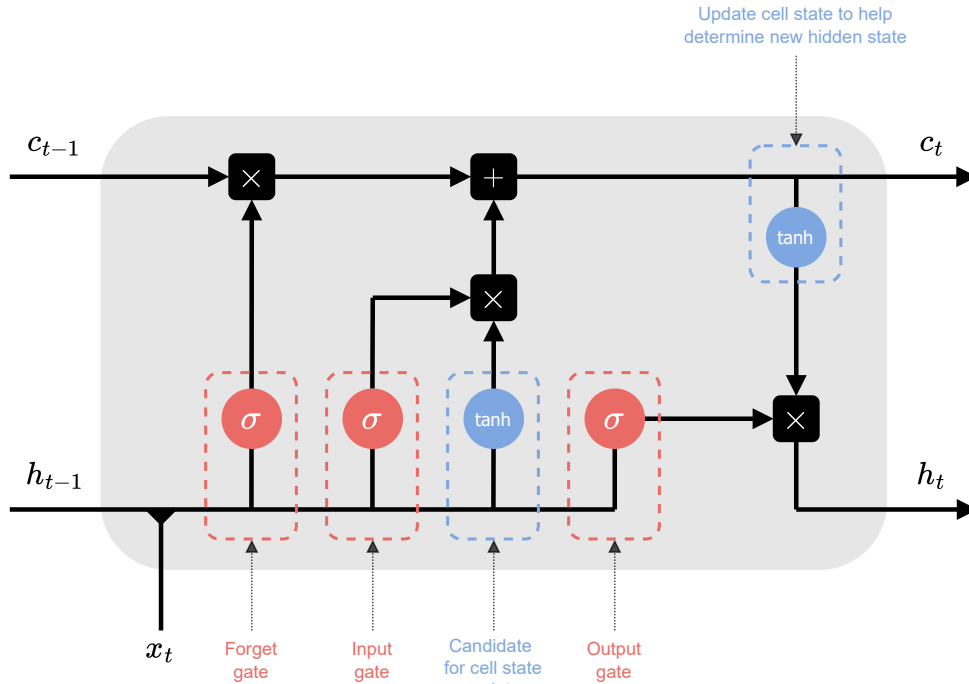


Figure 4.2: LSTM recurrent unit. Here c_t and h_t are the cell state and hidden state produced at time t , respectively. x_t is the model input. Image recreated from [here](#).

4.4.1 Long Short-Term Memory

The *Long Short-Term Memory* (LSTM) (Hochreiter and Schmidhuber, 1997) is a recurrent neural network (RNN) variant that is specifically designed for sequence data and aims to capture long-term temporal dependencies. LSTM introduces mechanisms to address the vanishing gradient problem encountered by traditional RNNs, enabling the network to better handle sequence data. The basic structure is illustrated in Figure 4.2. A critical component is the *memory cell*, which is responsible for storing and transmitting information by selectively remembering or forgetting certain information. This is achieved through the following three components:

- *Forget gate*: determines which information is deleted from the memory cell.
- *Input gate*: controls how much new information is added to the memory cell.
- *Output gate*: decides how much information is output from the memory cell

The forget gate and input gate adjust c_t , the state of the memory cell at each time step t , while the output gate is used together with the updated cell state to determine the new hidden state h_t , which contains the output that is passed to the next time step. Apart from the previous cell state c_{t-1} and hidden state h_{t-1} , the LSTM unit takes the current feature input of the model x_t as input. Mathematically, the gates, cell state, and hidden state are defined as follows:

$$\begin{aligned}
 f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
 i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
 o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
 \bar{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \bar{c}_t \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned} \tag{4.3}$$

Here, W_q and U_q are weight matrices and b_q a bias term, where the subscript q represents either the forget gate f , the input gate i , the output gate o , or the cell state c . The operator $\odot$ denotes the element-wise product, and σ and $\tanh$ are the sigmoid and tanh activation functions, respectively. Initially, $c_0 = 0$ and $h_0 = 0$.

4.4.2 LSTM Aggregator

We utilize the LSTM model as an aggregator in the GNN layers. As discussed in [Section 3.3.1](#), this was originally proposed by [Hamilton et al. \(2017\)](#) as an alternative to the default mean operator. However, instead of applying the LSTM to a random permutation of the node’s neighbours, we aim to capture the network dynamics. Since the timestamps of the edges impose a sequential ordering between the neighbours of each node, it would be unwise to disallow the LSTM to learn from such dependencies.

We therefore propose to aggregate messages from neighbours in increasing order of timestamp corresponding to the edge. More formally, we define the ordered sequence of neighbours of node v as $\mathcal{N}^T(v) = \langle u_1, \dots, u_n \rangle$, where $u_i \in \mathcal{N}$ is the i th neighbour of v in the sequence, and $\text{time}(\{u_i, v\}) < \text{time}(\{u_j, v\})$ for all $i < j$. Note that a neighbour u can occur multiple times in the sequence $\mathcal{N}^T$ when there are multiple edges between u and v . The aggregation formula then becomes:

$$\mathbf{a}_v^{(k)} = \text{LSTM}^{(k)}\left(\langle \mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}^T(v) \rangle\right). \quad (4.4)$$

Here, the sequence of neighbours $\mathcal{N}^T$ defines the input sequence in which the LSTM processes the neighbours’ embeddings. Similar to the MLPs in the GIN model detailed in [Section 3.3.2](#), we have multiple LSTMs, one for each layer k in the GNN. This allows the model to learn from different sequential orderings at each depth. The LSTMs are quite simple and only consist of a single layer of the same size as the GNN layers. The weights of the LSTMs are updated simultaneously with the other weights of the GNN during backpropagation.

4.5 The LAS-GNN Model

For detecting suspicious ML patterns, we primarily rely on the techniques that extend a fundamentally basic GNN. The overall structure of our GNN follows the inductive representation learning framework described in [Section 3.3](#), consistently sampling the full three-hop neighbourhood of seed nodes. We refer to our ultimate GNN model, which includes ego IDs, signed message passing, edge weight multiplication, and LSTM aggregation, as LAS-GNN (**L**STM-based **A**ggregation and **S**igned message passing).

Since we do not use actual data with relevant features and instead focus on minimal structural embeddings in synthetic networks generated by ourselves, all nodes are initialized with a single feature, the integer 1. For the ego IDs, we extend the node features by concatenating an additional 1 for seed nodes and 0 for non-seed nodes. Edges may include a weight or timestamp, depending on the task. Since we conduct mini-batch training, the data is divided into smaller batches, ready to be processed by the GNN layers.

An overview of the LAS-GNN architecture is provided in [Figure 4.3](#). Once the batches are created, they are passed through the graph neural network. The GNN consists of K layers corresponding to K rounds of message passing. In each layer, the previous node embeddings and the edge weights are fed into the aggregator. While we have illustrated the LSTM aggregator here, it is not mandatory for every layer. In preliminary experiments, we found that using a default ADD aggregator, which sums up the neighbour’s embeddings element-wise, can be advantageous in the first GNN layer (see [Section 6.1](#)). When we utilize the LSTM aggregator and combine it with signed message passing and edge weight multiplication, we get the aggregation and update

mechanisms described in Equation 4.5.

$$\mathbf{a}_v^{(k)} = \text{LSTM}^{(k)}\left(\langle \mathbf{d}_{u,v} \odot \mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}^T(v) \rangle\right), \quad (\text{AGGREGATE, 4.5a})$$

$$\mathbf{h}_v^{(k)} = \text{MEAN}\left(\mathbf{h}_v^{(k-1)}, \text{ReLU}(\text{NORM}^{(k)}(\mathbf{a}_v^{(k)}))\right), \quad (\text{UPDATE, 4.5b})$$

where

$$\mathbf{d}_{u,v} = \begin{cases} \text{Linear}(w_{u,v}) & u \in \mathcal{N}_{\text{in}}(v), \\ \text{Linear}(-w_{v,u}) & u \in \mathcal{N}_{\text{out}}(v). \end{cases} \quad (4.5c)$$

The UPDATE rule involves taking the mean between the previous embedding vector and the normalized result of the aggregation, to which a ReLU activation function is applied. The ReLU activation introduces non-linearity, helping to mitigate the vanishing gradient problem during training and enabling the neural networks to learn more complex relationships. We use *batch normalization* (Ioffe and Szegedy, 2015) as a normalization technique, stabilizing training across mini-batches by re-centring and re-scaling the outputs. This update rule is quite standard and is also used in the GIN implementation by Egressy et al. (2024).

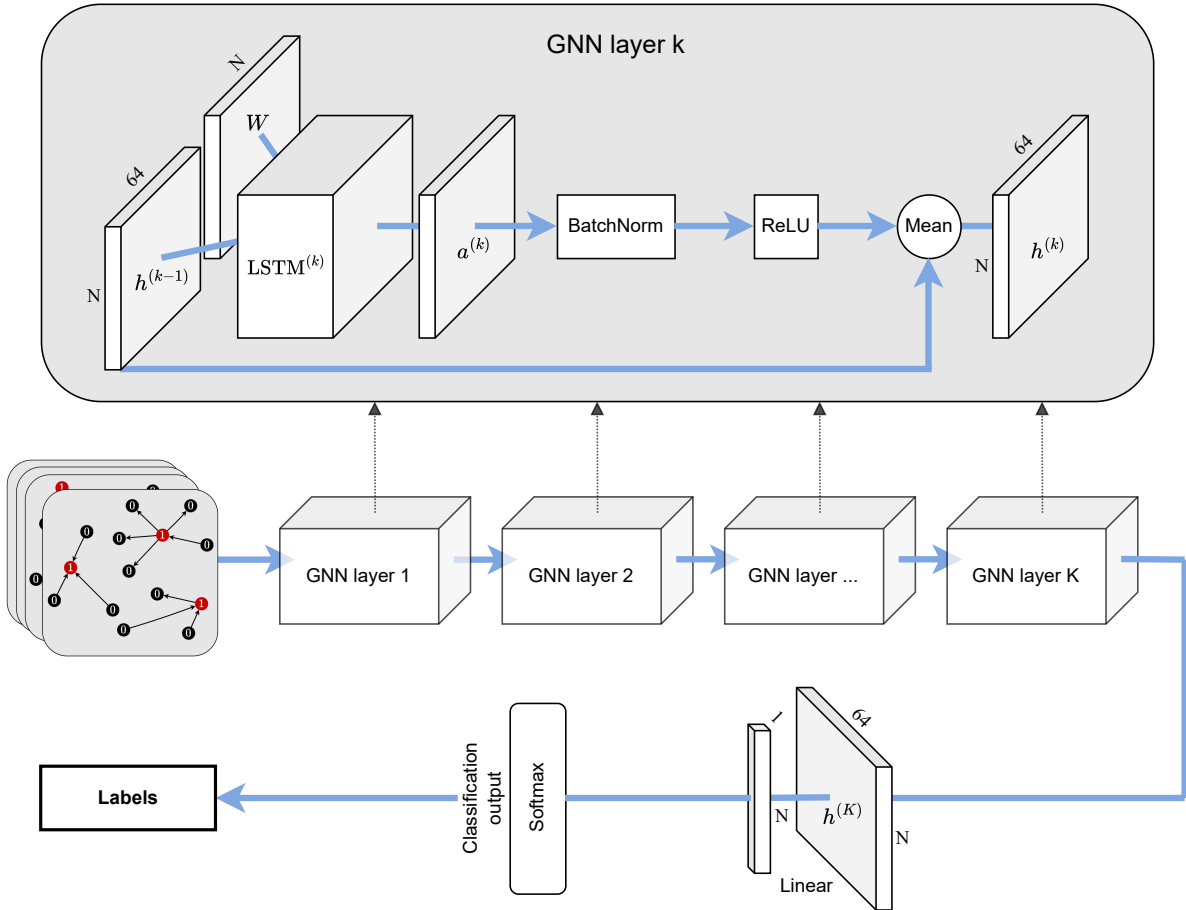


Figure 4.3: The architecture of LAS-GNN. The GNN consists of K layers, and a single layer with LSTM aggregation is detailed in the grey-filled box. We indicate embedding matrices of size $N \times 64$, where 64 is the embedding size and N is the number of nodes in the batched subgraphs. The $N \times N$ adjacency matrix W contains all edge weights w .

After K rounds of message passing, the final node embeddings are transformed by a linear layer to form raw output scores known as logits. The softmax activation function then normalizes the logits to a probability distribution over the predicted output classes. In our case, we predict the probability of the positive class, i.e., a node being the key node in a suspicious ML pattern. From these probabilities, we derive the predicted label (0 or 1) for each node by comparing it to a certain classification threshold (see [Section 5.4](#)).

4.6 Implementation

While our model is described conceptually, several practical elements lay at the foundation of the ego ID and signed message passing techniques. Due to certain limitations of existing MPNN implementations, specific design choices are essential for the model to function as intended. In this section, we highlight these issues and explain how to preprocess the input data accordingly.

4.6.1 Ego IDs

As stated in [Section 3.4.1](#), the ego ID is essentially a binary node feature set to 1 for certain nodes, namely the nodes we want to predict, and set to 0 for all other nodes. This approach fails when node embeddings are calculated for all nodes simultaneously since the GNN cannot differentiate between nodes that all have an ego ID of 1. However, when mini-batch training and inference are applied, only a small set of seed nodes are considered at a time. As long as the sampled neighbourhoods of these seed nodes do not overlap, a message tagged with an ego ID of 1 remains unique to its respective seed node within its neighbourhood. Even though the likelihood of overlapping neighbourhoods is low when the batch size is small relative to the size of the network, we ensure disjoint sampling of neighbourhoods: in case the extracted neighbourhoods of two seed nodes overlap, each neighbourhood is treated as a disjoint subgraph. Consequently, the graph in which the message passing is conducted may differ from the original input graph, since the former consists of multiple subgraphs and may contain some duplicate nodes. In that case, or when other adjustments have been made to the network, we refer to it as the *computation graph*. The creation of batches and assignment of ego IDs is illustrated in [Figure 4.4c](#).

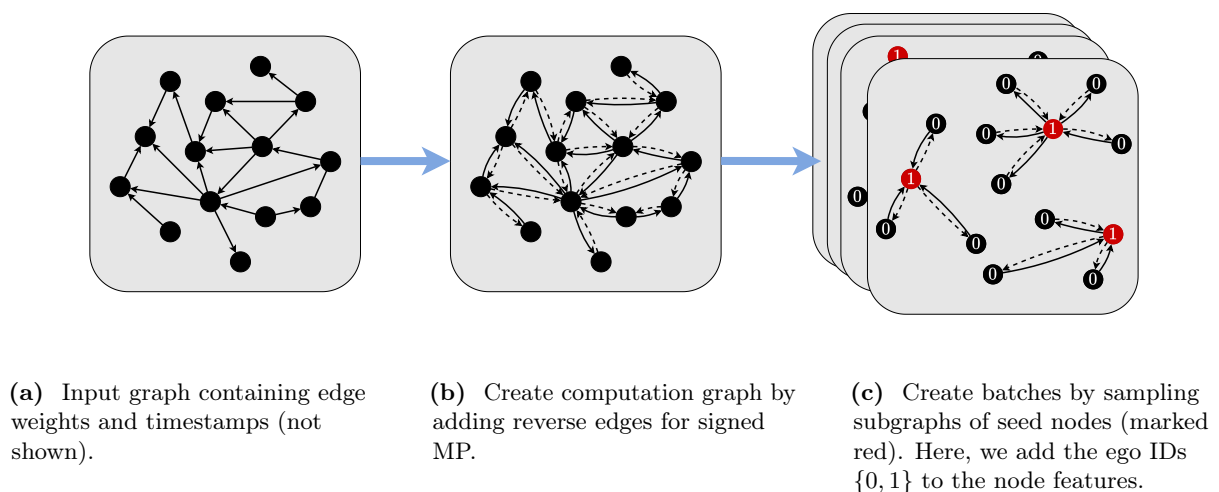


Figure 4.4: The preprocessing of the input graph in three steps. In (b), the reverse edges receive the negative weight and original timestamp of their corresponding original edges. For visibility, we only extract the one-hop neighbourhood of seed nodes in (c).

4.6.2 Signed Message Passing

In Equation 4.5, the signed message passing is formulated for the input graph. However, in practice, the actual implementation of MPNN architectures used (Fey and Lenssen, 2019) conducts message passing in a directional way, as described in Section 3.4.2. For bidirectional message passing, the network is converted to a seemingly undirected network by adding edges in opposite direction.

A similar approach is taken for heterogeneous and signed message passing, and the implementation of the latter is as follows: For each edge (u, v) with weight w and timestamp t in the input graph, we create an oppositely directed edge (v, u) with weight $-w$ and timestamp t and add it to the computation graph along with the original edge. While conducting message passing over the incoming edges (as done with directional message passing) in the computation graph, each node also receives messages from outgoing neighbours due to the artificially created reverse edges. This approach mimics bidirectional message passing while indicating the direction of the messages through the embedding of the signed weights.

Figure 4.4 describes the complete preprocessing of the input graph that besides the ego ID features and batch generation also includes adding the reverse edges to the computation graph in Figure 4.4b. As a result, signed message passing is now compatible with any spatial graph neural network implementation that supports message passing on directed networks.

Chapter 5

Experimental Setup

In this chapter, we discuss the details of how we design our experiments for assessing the performance of the LAS-GNN model and the GNN pattern finding techniques. We describe the synthetic network generation model used for creating our datasets in [Section 5.1](#). Furthermore, we discuss hyperparameter tuning and other training details in [Section 5.2](#). Model baselines are described in [Section 5.3](#) and description of appropriate evaluation metrics is given in [Section 5.4](#). Some brief implementation details regarding software and hardware can be found in [Section 5.5](#).

5.1 Synthetic Network Generation

Unfortunately, we do not have access to real-world transaction data, especially not including labels for reported money laundering cases. Hence, we do not rely on rich feature sets for our experiments. Instead, we turn to synthetic networks generated by ourselves, which allows us to have complete control over the datasets by adjusting specific network parameters and to generate as much data as needed.

5.1.1 Directed Watts-Strogatz Model

We adopt the *Watts-Strogatz model* (WSM) ([Watts and Strogatz, 1998](#)) as our preferred method of synthetic network generation. This random graph generation model produces graphs with small-world properties, including short average path lengths and high clustering. Such properties facilitate the emergence of numerous naturally occurring patterns in the network, such as short cycles and scatter-gather patterns. This ensures that patterns appear randomly, rather than being manually injected into the network post hoc, which risks skewing the random distribution and creating bias that affects detection algorithms, as simple indicators (such as node degree) may suffice to solve the task approximately ([Eggressy et al., 2024](#)). In addition, naturally occurring patterns will be overlooked when only injected patterns are labelled.

Another advantage of the WSM generation method is that it is fast and straightforward, unlike some alternative network generation methods, such as those within the preferential attachment family of models. However, a significant limitation is that it produces an unrealistic degree distribution, while real networks, including banking networks ([Saxena et al., 2021](#)), are often scale-free networks where the degree distribution follows a power law, enabling the presence of so-called *hubs* – nodes with high degree. Nonetheless, we argue that for researching the capabilities of pattern detection models, such hubs are not a necessity. Including hubs in the networks of our datasets would likely result in these nodes being involved in multiple patterns of varying types, complicating the investigation into what the model is learning and the interpretation of the results. Moreover, in real financial networks, it may not be so likely that large hubs serve as key nodes in money laundering schemes anyway, since these often correspond to companies not suspicious of money laundering ([Starnini et al., 2021](#)).

The WSM generation method can be described as follows. The inputs are the desired number of nodes N , the average degree k (assumed to be an even integer), and a parameter p_{rewire} , all satisfying $N \gg k \gg 1$ and $0 \leq p_{\text{rewire}} \leq 1$. Given these inputs, the model constructs an undirected graph with N nodes and $Nk/2$ edges in the following manner:

- (a) Construct a ring lattice comprising N nodes, where each node is connected to its $k/2$ immediate neighbours on both sides. More formally, suppose the nodes are labelled $0, \dots, N-1$, there is an edge $\{i, j\}$ if and only if

$$0 < |i - j| \bmod (N - 1 - \frac{k}{2}) \leq \frac{k}{2}.$$

- (b) For every node $i = 0, \dots, N-1$ in the graph, take every edge connecting i to its $k/2$ rightmost neighbours, that is every edge $\{i, j\}$ such that $0 < (j - i) \bmod N \leq k/2$, and rewire it with probability p_{rewire} . Rewiring is done by selecting a node u uniformly at random from all possible nodes and replacing edge $\{i, j\}$ with $\{i, u\}$, while avoiding link duplication and self-loops ($u \neq i$).

We extend the above algorithm with our so-called reciprocity parameter p_{recip} , where $0 \leq p_{\text{recip}} \leq 1$. This parameter is used to determine the direction of the edges in the random graph through an additional step:

- (c) For each edge $\{i, j\}$ independently with probability p_{recip} , convert it to a directed edge in both directions so that the graph includes both (i, j) and (j, i) . Conversely, with probability $1 - p_{\text{recip}}$, select a single direction: either (i, j) or (j, i) is included, with equal probability $(1 - p_{\text{recip}})/2$.

The final step introduces random edge directions in a straightforward manner to the otherwise undirected graph generated by the standard WSM. While we do not consider multi-edges, we do include the possibility of two-way edges using the reciprocity parameter. This enhances flexibility for the money laundering patterns to emerge, and it makes sense intuitively to allow for transfers in both directions between a pair of nodes. A small example network generated by steps (a), (b) and (c) is illustrated in Figure 5.1. Here, we can see how even a slight rewiring can transform the initially ordered lattice into a seemingly random graph.

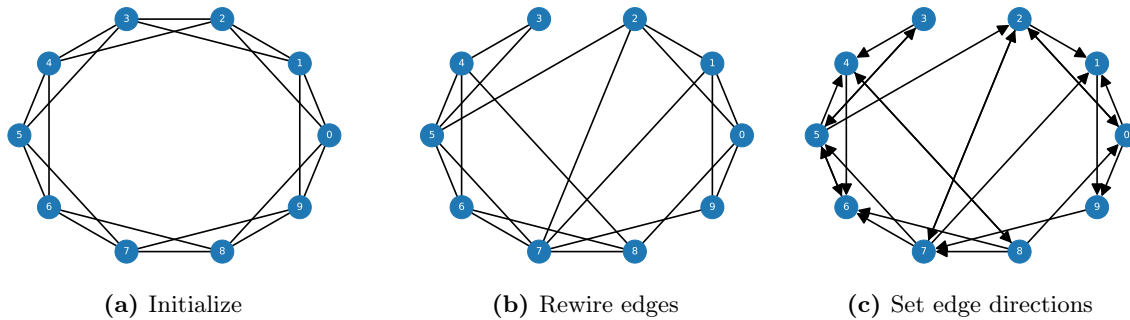


Figure 5.1: Example of the network generation steps from the directed Watts-Strogatz model with $N = 10$, $k = 4$, $p_{\text{rewire}} = 0.2$, and $p_{\text{recip}} = 0.2$.

5.1.2 Network Parameter Settings

To summarize, we have the following four network parameters that control the network generation described in the previous section:

- N : the total number of nodes
- k : the average degree
- p_{rewire} : the probability of rewiring an edge
- p_{recip} : the probability of edge reciprocation

To determine the right parameter settings for our experiments, we conducted a random search with various combinations of parameter values. Here, we primarily focussed on the frequency of different money laundering patterns naturally occurring within the network. Ultimately, we selected $N = 10000$, $k = 6$, $p_{\text{rewire}} = 0.4$, and $p_{\text{recip}} = 0.1$ as default values for generating the networks in our experiments. Note that there is no such thing as the “best” parameter setting here: the aim was to select some reasonable values that facilitate our experiments, and our chosen combination of parameter values is one of many combinations that could fulfil this.

To validate that our chosen parameter values produce appropriate networks for our experiments and to understand the impact of each network parameter on the number of patterns occurring in the data, we plot in Figure 5.2 the pattern occurrence percentage of each pattern for different

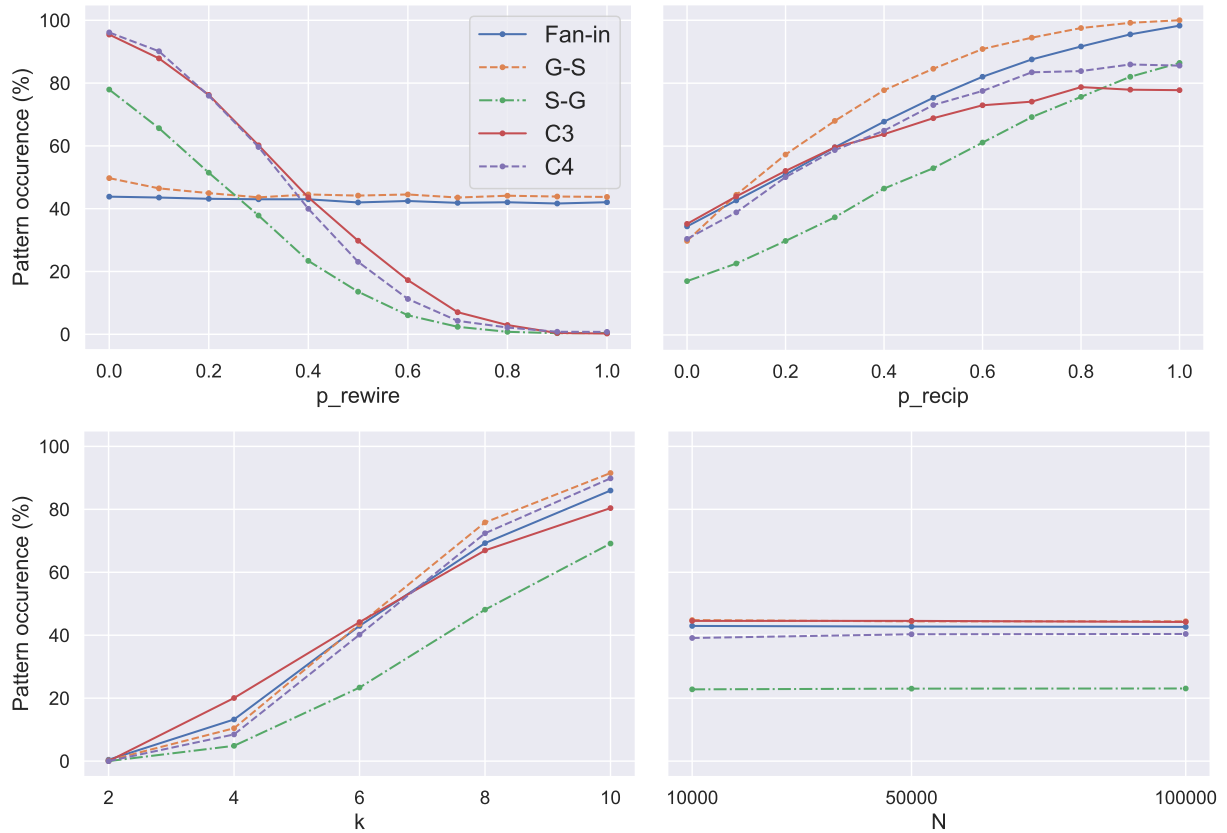


Figure 5.2: The effect of different WSM parameters on the frequency of ML patterns in the network. Here, we count the percentage of nodes involved as a key node in a pattern. In each subplot, a single parameter is varied (x-axis), while the other parameters remain fixed at their default value: $N = 10000$, $k = 6$, $p_{\text{rewire}} = 0.4$, and $p_{\text{recip}} = 0.1$.

parameter values. With pattern occurrence, we refer to the percentage of nodes involved as a key node in a basic money laundering pattern. We omitted some patterns (fan-out, bipartite-clique) from the plots due to their distribution being nearly identical to other patterns (fan-in, scatter-gather).

In the top left figure, we observe that distorting the network by rewiring edges rapidly decreases the number of scatter-gather and Cycle patterns. When $p_{\text{rewire}} = 0$, these patterns are ubiquitous in the data due to the initial setup where each node links to its direct neighbours. In contrast, fan-in and gather-scatter patterns depend on the degree of nodes, which remains unchanged when the network is randomized. Increasing p_{recip} quickly inflates the occurrence of all patterns, so a low value seems appropriate. Moreover, the bottom right figure shows that the occurrence percentage remains unaffected by the number of nodes in the network for all patterns, indicating that the number of patterns in the network grows proportionately with N . This is beneficial as it allows us to adjust the network size while maintaining the same class imbalance.

Finally, we selected the network parameter values as specified to ensure a sufficient number of patterns for the model to learn from while keeping the proportion of nodes involved as key nodes in a ML pattern below 50%. This yields an initial class imbalance that gradually increases once we introduce weighted and temporal patterns.

5.1.3 Edge Weights and Timestamps

When we introduced the money laundering patterns in [Section 4.1](#), we described the weighted and temporal patterns as patterns that were constrained by the edge weights and timestamps, respectively. To be more specific, we use the following definitions in our tasks:

- **Weighted pattern:** We consider a pattern a weighted pattern if and only if all the edge weights within the pattern exceed a certain threshold r .
- **Temporal pattern:** We consider a pattern a temporal pattern if and only if the edge timestamps are sequentially ordered for the intermediate nodes, ensuring that an intermediate node first receives money and then sends money.

While we can apply the weighted pattern definition to all basic ML patterns, the temporal definition is only applicable to patterns that contain intermediate nodes: the gather-scatter, scatter-gather, and simple cycle patterns. For a temporal gather-scatter pattern, all edges of the fan-in have timestamps earlier than those of the fan-out. For a temporal scatter-gather pattern, we have that the incoming edge of the intermediate nodes must have an earlier timestamp than the corresponding outgoing edge. The same holds for a temporal simple cycle pattern, only there must be a single node with the temporal order of its two pattern edges reversed: this node initiates by sending first and completes the cycle by receiving last. We designate this node as the key node to be identified in temporal tasks. Examples of weighted and temporal patterns are illustrated in [Figure 5.3](#).

For adding edge weights to our synthetic network, we assume that they are generally unrelated and apply a uniform random distribution $U(0, 1)$ to generate them independently. To simulate that edges within a suspicious ML pattern are stronger, we also apply a uniform random distribution, but we shift the range of the weights to be higher. The parameter $0 \leq r \leq 1$ serves as a threshold value that determines the minimum weight of the edges in a suspicious money laundering pattern, meaning we apply a uniform random distribution $U(r, 1)$.

We convert the static network to a dynamic setting by generating edge timestamps. Since only the sequential order of edges is relevant for the LSTM aggregation, we take the straightforward approach of randomly shuffling the m edges of the graph and assigning a unique timestamp in that order from the set $\{1, \dots, m\}$. As a result, some ML patterns occurring in the data are temporally ordered, while others are not.

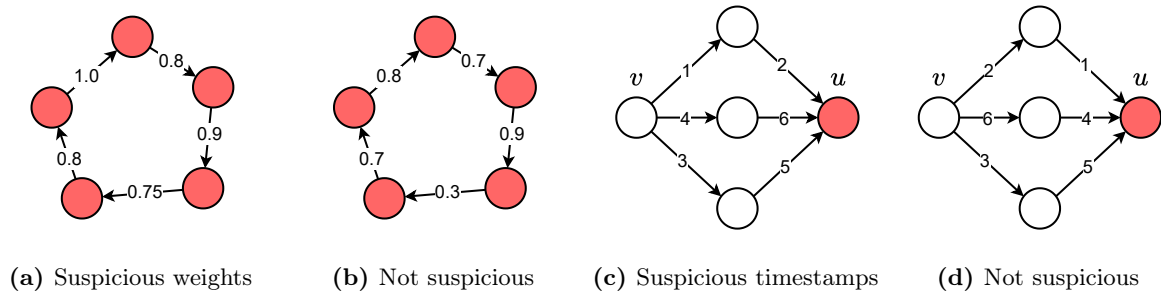


Figure 5.3: Example of weighted simple cycle patterns and temporal scatter-gather patterns. The weighted cycle in (a) is suspicious since all weights exceed the threshold $r = 0.7$, while the cycle in (b) has an edge with weight lower than the threshold, breaking the suspicious pattern. The pattern in (c) is suspicious, since all three intermediate nodes first receive and then send. In (d) however, two intermediate nodes first send and only receive later. Note how the edge ordering for v and u does not matter here.

5.1.4 Ground Truth

Apart from generating the synthetic networks, we need ground truth labels that specify whether a node is part of a suspicious ML pattern. Such labels serve two purposes: 1) to supervise the learning during training and 2) to evaluate the model’s performance.

Our approach to establishing ground truth is as follows: Given a generated input network, we employ an exhaustive search to determine the patterns occurring in the graph. For instance, to detect cycles, we can apply any cycle detection algorithm and adjust it to only return simple cycles. For scatter-gather patterns, we designate any pair of nodes as the source and sink nodes and verify whether the set of incoming neighbours of the sink node shares more than one node with the set of outgoing neighbours of the source node. Once identified, we classify the key node of a pattern as suspicious (1), while all other nodes are marked non-suspicious (0). While the exhaustive search for these basic patterns is quite efficient, we only use this for determining the ground truth labels. The learning algorithm does not have access to this information, and so the goal remains to let the GNN learn these patterns solely from the data.

For classifying weighted and temporal patterns, we simply check for each pattern found whether the edge weights or timestamps match the definitions given in [Section 5.1.3](#). In the case of weighted patterns, a pattern is deemed suspicious when all weights within it exceed the threshold r . Otherwise, the key node is labelled 0. This criterion applies to patterns intentionally designed to be suspicious using the $U(r, 1)$ distribution as well as those where all weights accidentally exceed r despite being generated from $U(0, 1)$.

In tasks with both weighted and temporal patterns, we consider a pattern suspicious when it is suspicious according to both the weighted and temporal definitions.

5.2 Model Training

For our LAS-GNN model, we have the following hyperparameters that control the model’s learning process:

- *learning rate*: the step size at which the optimizer makes updates to the model’s weights. Default values are $[0.01, 0.001, 0.0001]$.
- *embedding size*: the length of the embedding vector. Since the same dimensions are used for all hidden layers in the GNN, including the single-layer LSTM aggregators, this parameter also sets the number of neurons in a single layer. Default values are $[16, 32, 64, 128, 256]$.

- *batch size*: the number of seed nodes to consider per mini-batch. Default values are [16, 32, 64, 128, 256].

We conducted a grid search with these default hyperparameter values on networks of 10K nodes considering both weighted and temporal patterns. Preliminary investigations indicated that a **learning rate of 0.001**, **embedding size of 64**, and a **batch size of 32** provide satisfactory results across all tested patterns. These values are fixed for all subsequent experiments unless otherwise specified. While further tuning, using an annealing learning rate or adding regularization techniques such as node drop-out, is possible, we do not see any gain to be achieved by doing so.

For stochastic optimization, we employ the *Adam optimizer* (Kingma and Ba, 2014) alongside a standard binary cross-entropy loss function. To address the data imbalance, we use the ratio of positive class labels out of the total number of samples as a weight in the loss calculation. Each model is trained for a maximum of 100 epochs unless specified otherwise. We assume convergence and apply early stopping once the validation loss does not improve for 20 epochs.

Another parameter that needs to be determined is the number of GNN layers, K . However, this is pattern-dependent and tested explicitly in Section 6.1. The same holds for the number of LSTM aggregators relative to K .

5.3 Compared Models

In our experiments, various GNN models, defined by the selection of GNN techniques incorporated, are compared. First, we explore the different types of message passing and investigate how many GNN layers are necessary for initial pattern detection in preliminary experiments. Here, we use the most basic GNN model without any techniques for weighted or temporal patterns, which we simply refer to as *GNN*.

We also include the *GIN* model of Egressy et al. (2024) in the unweighted non-temporal task. As described previously, the GIN model can be extended to include edge features to deal with weights and/or timestamps. We refer to this model as *E-GIN* (Extended-GIN) and include it in the weighted and temporal tasks. The timestamp feature is L_2 normalized since using absolute timestamp values results in worse performance. The MLP used is a two-layer neural network with 64 nodes in each layer. The ϵ -parameter is set to 0.

The above models all use the default ADD aggregator that applies an element-wise sum to the embeddings of a node’s neighbours. Our LAS-GNN model uses LSTM aggregation instead, where the LSTMs are applied to the sequence of neighbours sorted by edge timestamp (only possible in temporal tasks). Additionally, LAS-GNN utilizes edge weight multiplication, where the weights are set to 1 in unweighted tasks. Furthermore, we include two LAS-GNN variants as baselines. *Random* LAS-GNN applies the LSTM aggregator to a random permutation of the nodes, similar to the proposed aggregator in GraphSAGE (Section 3.3.1). We use this model in non-temporal tasks, and as a baseline in the unweighted temporal task. Another variant is *Hetero*

Table 5.1: Overview of the models used in our experiments.

Model	Ego ID	MP type	Weighted	Temporal	Aggregator
GNN	✓	<i>any</i>	✗	✗	ADD
GIN	✓	hetero	✗	✗	ADD
E-GIN	✓	hetero	edge feature	edge feature	ADD
LAS-GNN	✓	signed	weight mult.	edge sorting	LSTM
- Random	✓	signed	✗	✗	LSTM
- Hetero	✓	hetero	✗	edge sorting	LSTM

LAS-GNN, which does utilize the sorted LSTM aggregator, but due to heterogeneous message passing applies it to the incoming and outgoing edges separately.

Table 5.1 presents an overview of the models. The hyperparameter settings of all described benchmark models are the same as LAS-GNN, detailed in the previous section.

5.4 Evaluation Metrics

After we train a graph neural network model on a synthetic network, we compare its node-level predictions to the ground truth labels. For validation and evaluation, we apply the trained model on two separate networks (randomly generated with the same network parameters) to ensure no data leakage and correct inductive learning. In each experiment, we run this process multiple times, each on a distinct network, and average the results. In this section, we discuss what evaluation metrics are appropriate for assessing the performance of the models and the pattern finding techniques.

5.4.1 Confusion Matrix

Since there are only two classes, namely suspicious and non-suspicious (normal), we have a binary classification problem. The results can be reported in a 2×2 *confusion matrix*. Here, we denote the positive class (suspicious) with label 1 and the negative class (non-suspicious) with label 0. For each combination of the actual label and the predicted label, the number of observations is reported in the corresponding cell, comprising the True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

		Predicted	
		1	0
Actual	1	<i>TP</i>	<i>FN</i>
	0	<i>FP</i>	<i>TN</i>

5.4.2 Accuracy, Precision, Recall, F_1 -score

Now that we have defined the confusion matrix, we can derive other performance metrics using these values. A simple and well-known metric is the *accuracy*, which measures the proportion of correct predictions out of all predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

However, in the context of AML, accuracy is not a suitable metric (He and Garcia, 2009). The reason for this is that the accuracy does not distinguish between the numbers of correctly classified examples of different classes. Since the data in AML is often imbalanced, the metric can be misleading. In our case, the positive class is often the minority class, and it is most critical to detect these suspicious cases. However, when the imbalance is severe, a model predicting every observation to be negative achieves a high accuracy!

More insightful metrics are the *precision* and *recall*, calculated as follows:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}.$$

The precision, or positive predictive value, is defined as the proportion of positive class predictions that were correct. The recall, also known as the sensitivity or the true positive rate, is defined as the proportion of actual positive class samples that were identified as such by the model.

We can combine these two competing metrics into a single evaluation metric known as the F_1 -score:

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times TP}{2 \times TP + FP + FN}.$$

The F_1 -score is the harmonic mean of the precision and recall and thus represents both equally in one metric. Its highest possible value is 1.0, indicating perfect precision and recall, and the lowest possible value is 0.0 when either precision or recall is zero. The score is centred on the positive class and is thus a more appropriate measure than the accuracy. In [Chapter 6](#), we use the F_1 -score to assess whether a GNN model with specific techniques can detect ML patterns almost perfectly (F_1 close to 1.0) or not. We also provide F_1 scores in tasks where the data classes are balanced due to under-sampling the majority class.

5.4.3 Precision-Recall Curve, Average Precision

While we aim for both high precision and high recall, there is a trade-off between the two metrics. Generally, improving precision leads to a decrease in recall and vice versa. To increase precision, a model may become more conservative in making positive predictions. While this would result in fewer false positives, it comes at the cost of potentially missing some true positives, thus reducing recall. To enhance recall, on the other hand, a model may become more liberal in making positive predictions. While this would capture more true positives, it may also increase the number of false positives, which leads to reduced precision. In a practical context, the right balance between the two metrics depends on the specific application.

A technique for finding a desired balance between precision and recall is to shift the model's classification threshold, i.e. the cut-off point used to assign a specific predicted class for each example. The default classification threshold of 0.5, which predicts class 1 if the prediction probability exceeds 0.5 and 0 otherwise, may not represent an optimal interpretation of the predicted probabilities. As mentioned, this can be the case when the class distribution is severely skewed or when one type of misclassification is more critical than another type of misclassification. Since we do not want to make assumptions about the desired balance between precision and recall, we plot the trade-off for different classification thresholds. In a so-called *Precision-Recall curve* (PR-curve), we plot the recall along the horizontal axis and precision along the vertical axis. Each point on the curve is a combination of precision and recall values that corresponds to a specific classification threshold. A skilful model is represented by a curve that bows towards a coordinate of (1, 1). A no-skill classifier will be a horizontal line on the plot with a precision proportional to the number of positive examples in the dataset. An illustration is shown in [Figure 5.4](#).

We can compute the *area under the precision-recall curve* ($PR\ AUC$) as a metric that summarizes the performance of the classifier across different classification thresholds. The area under the curve can be approximated with the trapezoidal rule. However, the use of linear interpolation can be too optimistic ([Davis and Goadrich, 2006](#)), and so instead, we use the *average precision* (AP), which summarizes a precision-recall curve as the weighted mean of precision scores achieved at each threshold, with the increase in recall from the previous threshold used as the weight ([Pedregosa et al., 2011](#)):

$$AP = \sum_n (R_n - R_{n-1}) P_n,$$

where P_n and R_n are the precision and recall at the n -th threshold.

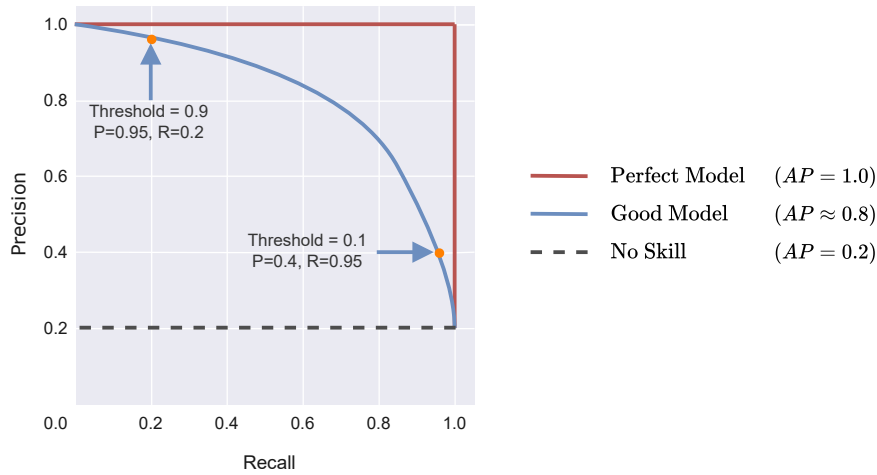


Figure 5.4: Example of a plot with three different precision-recall curves: a perfect model, a good model, and a no-skill baseline proportional to the imbalance ratio (in this case 0.2). Highlighted are two threshold points and their corresponding precision and recall values.

5.5 Technical Specifications

Most of the code for the experiments is written in `Python`. The synthetic network generation is implemented using `NetworkX` library (Hagberg et al., 2008). We use `PyTorch` and specifically `PyTorch Geometric` (Fey and Lenssen, 2019) and its additional dependencies for the graph neural network implementations. For statistical analysis and evaluation, we rely on `Scikit-learn` (Pedregosa et al., 2011). All of our code can be found on GitHub¹, along with a complete list of dependencies and installation instructions.

All experiments were run on a server running Ubuntu 22.04 LTS whose specs are Intel(R) Xeon(R) Gold 6238R CPU @ 2.20GHz, Nvidia(R) A100 PCIe 40 GB GPU, and 256GB of RAM.

¹Our GitHub repository can be found here: <https://github.com/Sverlaan/LAS-GNN>.

Chapter 6

Results

In this chapter, we present the results of our experiments. After some preliminary experiments in [Section 6.1](#), we proceed in a step-by-step manner by introducing weighted patterns, temporal patterns, and their combination, in separate tasks. In these sections, we specifically focus on gather-scatter patterns, scatter-gather patterns, and simple cycles of length 3. As we search for more specific patterns with each task, the class imbalance becomes more pronounced. Finally, we explore the case of longer temporal cycle patterns in [Section 6.6](#).

6.1 Preliminary Experiments

First, we explore the different types of message-passing with and without ego IDs to evaluate their effectiveness. Moreover, we investigate the number of GNN layers required for initial pattern detection as well as the number of LSTM layers necessary for temporal pattern detection.

6.1.1 Edge Directionality

To highlight their relative importance, we compare the most basic GNN model with the different types of message-passing schemes ([Section 3.4.2](#)), with and without the addition of ego IDs ([Section 3.4.1](#)). The results for the basic ML patterns are presented in [Table 6.1](#).

As the results confirm, bidirectional message-passing makes it impossible to detect any of the basic ML patterns. The reason for this is that all the patterns are defined by directed subgraphs, and bidirectional MP does not account for the directionality. The addition of ego IDs does not seem to make any significant improvement. When we look at the other message-passing schemes, we see that a fan-in pattern is always perfectly detectable. Neither heterogeneous nor signed

Table 6.1: *F1* scores averaged over 5 runs on different WS networks with 10K nodes. Each GNN model consists of 4 layers and is trained for 10 epochs. Standard deviations are all ≤ 0.03 and are omitted in favor of readability. Perfect detection results are highlighted.

Model	Fan-in	Fan-out	G-S	S-G	B-C	C3	C4
Bidirectional	0.64	0.67	0.73	0.48	0.47	0.71	0.64
+ EgoIDs	0.64	0.65	0.73	0.49	0.49	0.73	0.68
Directional	1.0	0.63	0.78	0.57	0.55	0.67	0.61
+ EgoIDs	1.0	0.68	0.80	0.62	0.58	1.0	0.75
Heterogeneous	1.0	1.0	1.0	0.63	0.64	0.85	0.70
+ EgoIDs	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Signed	1.0	1.0	1.0	0.64	0.63	0.86	0.71
+ EgoIDs	1.0	1.0	1.0	1.0	1.0	1.0	1.0
<i>Imbalance (%)</i>	42.5%	42.5%	45.2%	23.8%	23.7%	45.7%	40.7%

message passing makes a difference in this case, nor does the addition of ego IDs. For detecting fan-ins, the GNN must essentially count the in-degree of a node for which directional message passing is already sufficient. Contrastingly, this is not the case for fan-out patterns; for counting the out-degree of a node, either heterogeneous or signed MP is required. Since a gather-scatter pattern is essentially a combination of a fan-in and fan-out pattern, it comes as no surprise that it is only detectable in case of heterogeneous or signed message passing as well.

For scatter-gather patterns, the results clearly indicate that both heterogeneous or signed MP and ego IDs are necessary for detection. While F1 scores improve slightly with each more advanced MP scheme, this alone is insufficient for pattern identification. Although the ego ID does not significantly impact directional MP, it has a substantial effect on heterogeneous and signed MP, raising the F1 score from around 0.64 to a perfect 1.0. The same results are observed for the bipartite-clique pattern. Similarly, the cycle patterns are also perfectly detectable in case ego IDs are used in conjunction with either heterogeneous or signed MP. When we compare the cycles of length 3 with cycles of length 4 in the alternative cases, we see that C4 consistently has a lower F1 score. This indicates that longer cycles are generally harder to detect. This is especially evident in the case of directional MP + EgoIDs, where C4 has an F1 score of 0.75, while C3 detection is flawless.

These experiments reproduce the results of [Egressy et al. \(2024\)](#), demonstrating that heterogeneous message passing combined with the addition of ego IDs to the node features is essential to correctly identify the basic money laundering patterns. Furthermore, our alternative approach of signed message passing seems to be just as effective, achieving perfect detection results in directed graphs with reciprocating edges.

From now on we will always conduct signed message passing and add ego IDs, unless specified otherwise. The biggest exception is the (E)-GIN model, which always performs heterogeneous message passing.

6.1.2 Number of GNN Layers

The number of GNN layers used in the previous experiments is $K = 4$, which intuitively appears to be the minimum number of layers required for detecting certain ML patterns. For instance, in

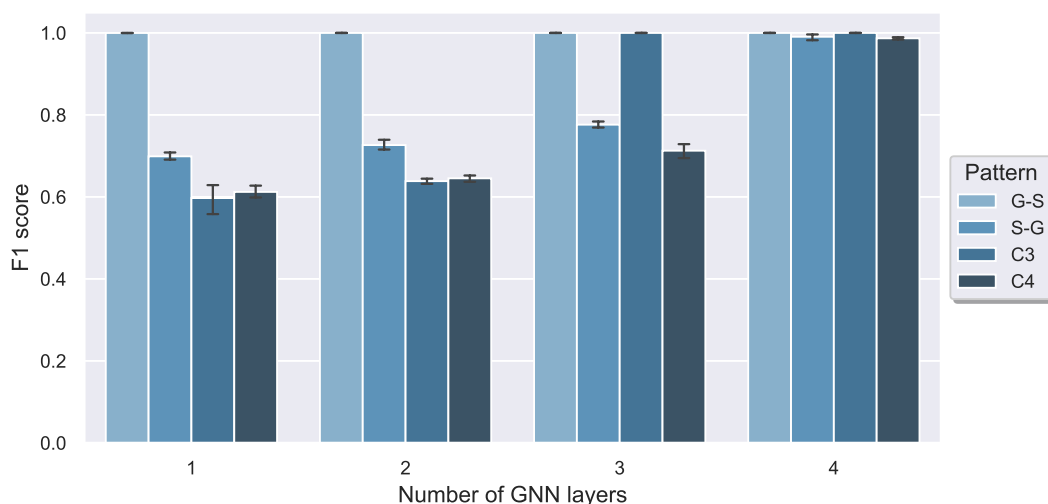


Figure 6.1: F1 scores achieved by GNN (signed MP + ego IDs) for different number of GNN layers. Results are averaged across 5 different runs and error-bars with the standard deviation are shown. This task is balanced, meaning we sampled 2000 key pattern nodes and an equal number of non-pattern nodes at random.

a C4 pattern, the ego ID must traverse four edges to return to its original seed node. Similarly, a scatter-gather pattern involves at least two distinct paths of length two between the source and sink nodes. Therefore, four rounds of message passing are required, corresponding to four GNN layers that together enable message passing at a depth of four from the seed node. To experimentally verify this, we plot the F1 scores achieved by the GNN with signed message passing and ego IDs for each number of GNN layers between 1 and 4. Results are shown in Figure 6.1.

We observe that only a single GNN layer is needed to detect the gather-scatter pattern (and, by extension, the fan-in and fan-out patterns). Even with additional layers, the detection of these local patterns stays robust, showing no decline in performance. For simple cycle patterns, the number of GNN layers required indeed corresponds to the length of the cycle: C3 and C4 are detectable with three and four rounds of message passing, respectively. Similarly, at least four GNN layers are necessary for scatter-gather detection. The same holds for the bipartite-clique pattern, which is not included in the figure due to the aforementioned similarity.

Since four GNN layers is sufficient to detect all basic ML patterns mentioned, we will always use this number of layers in future experiments, unless specified otherwise.

6.1.3 Number of LSTM layers

Now that we know four GNN layers are sufficient to detect the basic ML patterns, the next question is how many of these four layers must rely on LSTM aggregation to detect unweighted temporal patterns. Figure 6.2 presents average F1 scores of LAS-GNN for different numbers of LSTM layers out of the total four. When there are zero LSTM layers, all four GNN layers use ADD aggregation. LSTM layers are introduced from the final layer backwards, so for i LSTM layers, the final i layers of the GNN are replaced with LSTM aggregation.

The figure shows that having such an aggregator in the final layer is sufficient for detecting temporal gather-scatter patterns. For both the scatter-gather and C3 pattern, we achieve an F1 score close to 1.0 when the final two layers are LSTM layers, indicating that LAS-GNN can

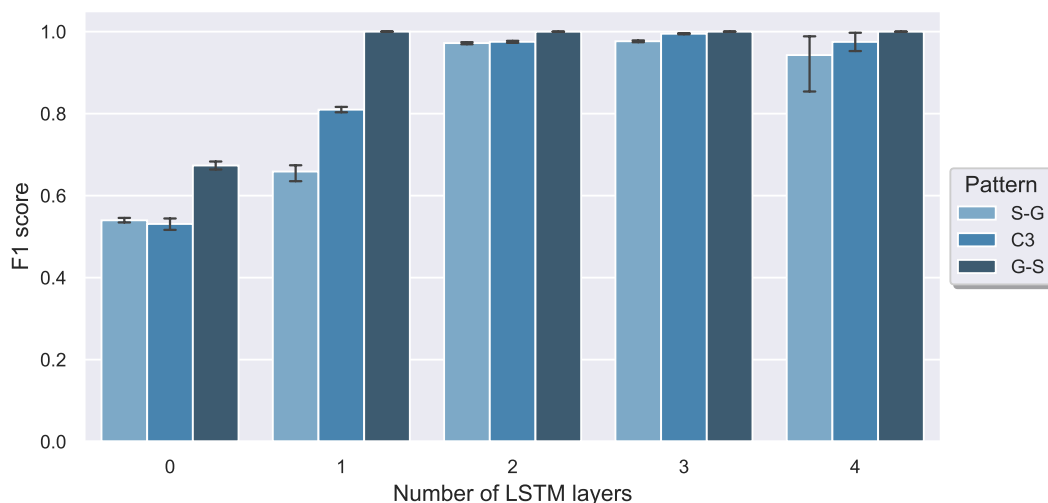


Figure 6.2: Average F1-scores achieved by LAS-GNN for unweighted temporal pattern detection with different number of LSTM aggregation layers. Here, we investigate the number of LSTM layers out of four GNN layers total. Alternative layers use default ADD aggregation. Results are averaged across 5 runs. This task is balanced: we sampled 2000 temporal pattern nodes and an equal number of pattern nodes that are not temporally ordered at random from a 30K network.

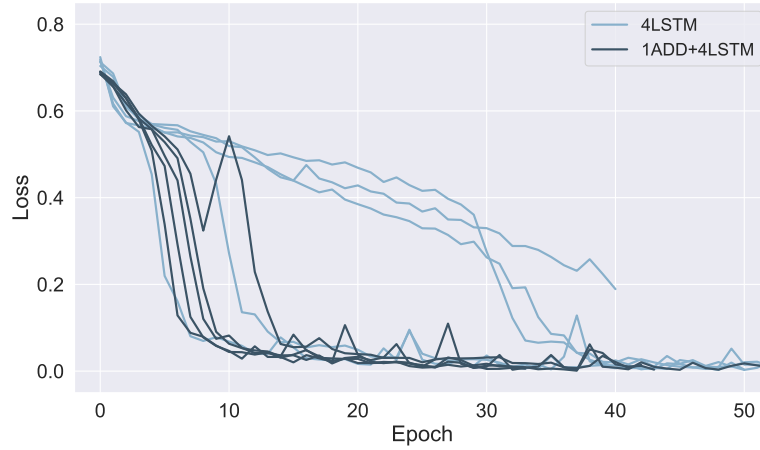


Figure 6.3: Training loss per epoch for LAS-GNN with four LSTM layers and LAS-GNN with an additional ADD layer in front. Here, we focus on unweighted temporal scatter-gather patterns.

effectively distinguish the temporally ordered patterns from patterns that are not. Adding a third LSTM layer improves performance further, although slightly. However, when all four GNN layers use LSTM aggregation, the performance unexpectedly drops, most notably for the scatter-gather pattern. It raises the question of whether the decrease in performance is due to having too many LSTM layers or if it is specifically because all the layers are LSTM layers.

To investigate this, we track LAS-GNN’s training loss progression for two configurations: 1) with all four layers using LSTM aggregation and 2) with one initial ADD aggregation followed by four LSTM layers. In [Figure 6.3](#), we observe that the 1ADD+4LSTM model converges after around ten epochs, whereas the 4LSTM model takes almost 40 epochs to converge in three out of five runs. One of these runs converges prematurely at epoch 40 due to early stopping, indicating that the validation loss (not shown) had not improved after epoch 20. From this, we conclude that a single ADD aggregation in the first GNN layer helps stabilize performance and speeds up convergence.

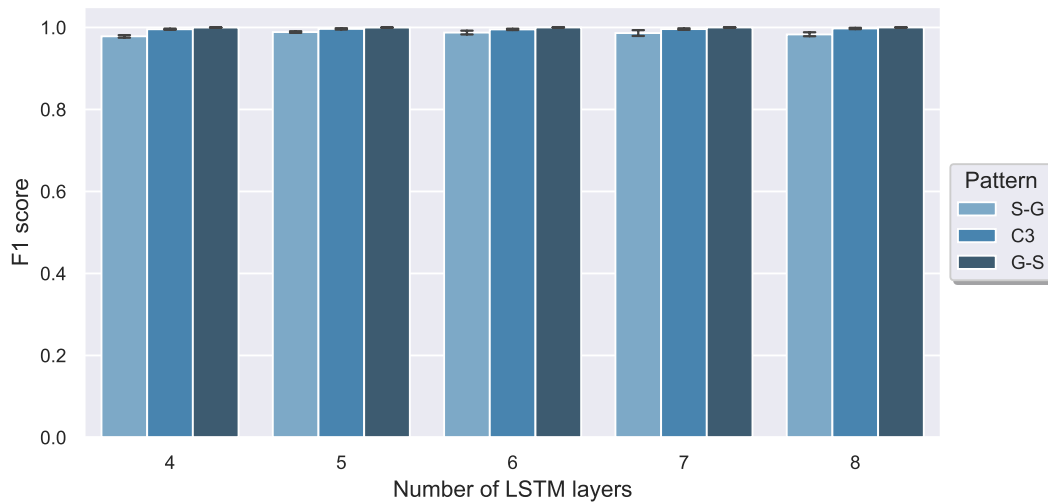


Figure 6.4: Average F1-scores achieved by LAS-GNN for unweighted temporal pattern detection with different number of LSTM aggregation layers. In this case, the initial layer uses ADD aggregation, followed by the specified number of LSTM layers.

In Figure 6.4, we observe that increasing the number of LSTM layers following an initial ADD layer does not impair the model’s ability to detect basic temporal patterns, even when it uses more layers than necessary. LAS-GNN seems robust to an abundance of LSTM layers and can manage the additional noise well. Consequently, smaller temporal patterns can still be detected, even when the model is aimed at larger patterns that require the learning of sequential ordering at greater depth.

In future experiments, our GNN models with LSTM aggregation always consists of one ADD layer followed by three LSTM layers, unless specified otherwise.

6.2 Unweighted Non-temporal Detection

In the preliminary experiments, we mostly focused on the basic GNN model. However, for weighted and temporal detection tasks, more advanced graph neural network models are required: E-GIN and LAS-GNN. To make sure that our LAS-GNN model does not lack in performance against Egressy et al. (2024)’s GIN model for initial pattern detection, we briefly compare the two on the unweighted non-temporal task. Since we do not have edge weights or timestamps, we simply compare the GIN model with the Random LAS-GNN model, where all edge weights are set to 1 and the LSTM aggregator of Random LAS-GNN is applied to a random permutation of neighbours.

From the perfect results in Table 6.2, we can conclude that using LSTMs instead of MLPs does not deteriorate initial pattern detection capabilities, even when the task does not require learning any sequential order.

Table 6.2: *F1* scores averaged over 5 runs on networks with 10K nodes. Each model is trained for 10 epochs. Here, the LSTM of Random LAS-GNN aggregates embeddings in arbitrary order.

Model	Fan-in	Fan-out	G-S	S-G	B-C	C3	C4
GIN	1.0	1.0	1.0	1.0	1.0	1.0	1.0
Random LAS-GNN	1.0	1.0	1.0	1.0	1.0	1.0	1.0
<i>Imbalance (%)</i>	<i>42.5%</i>	<i>42.5%</i>	<i>45.2%</i>	<i>23.8%</i>	<i>23.7%</i>	<i>45.7%</i>	<i>40.7%</i>

6.3 Weighted Non-temporal Detection

When we add edge weights to the network, the GNN model must not only identify the structural topology that the key nodes of distinct patterns of the same type have in common, but it must also relate this to the weights of the edges. In Section 4.3, we proposed to handle this with edge weight multiplication in the aggregation of neighbour embeddings, which can easily be integrated with signed message passing. On this weighted detection task, we compare the following three models: 1) The basic GNN model extended with edge weight multiplication, 2) Random LAS-GNN, which uses in addition to the edge weight multiplication a randomized LSTM aggregator, and 3) E-GIN with the weights as edge features. For this task, we use the same 10K networks from the initial pattern detection, but approximately half of the patterns are modified to contain edge weights above r only. The precision-recall curves for different thresholds r in scatter-gather pattern detection are shown in Figure 6.5.

We observe the GNN with edge weight multiplication performs quite well for the higher threshold values r . However, as r decreases and the range of the weights in a suspicious ML pattern increases, the model starts to misclassify more nodes. While an average precision of 0.85 for $r = 0.5$ is still fine, it does not compare well with the other two models. Both Random

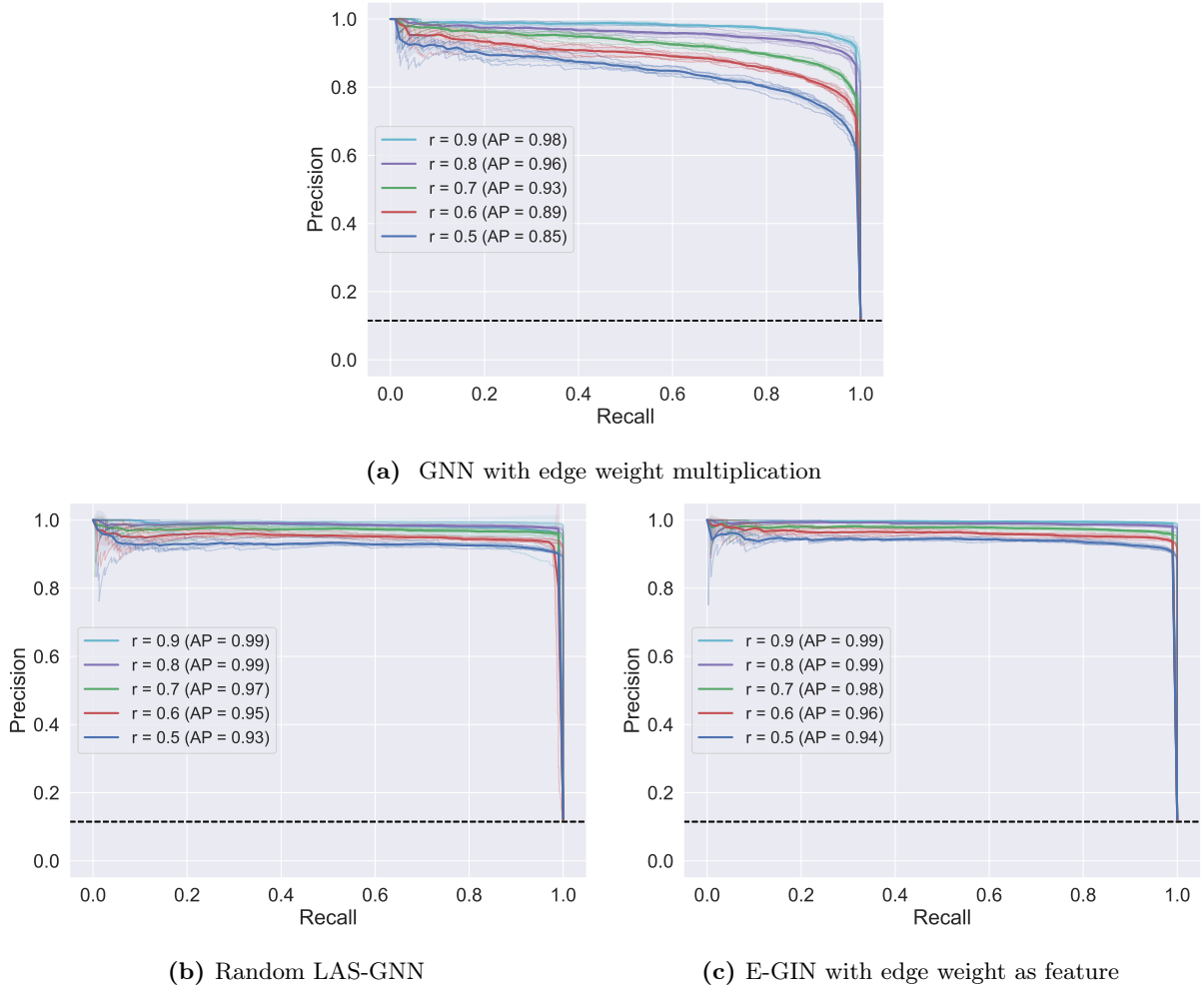


Figure 6.5: Precision-Recall curves for weighted scatter-gather pattern detection on networks with 10K nodes. For each weight threshold value r , we plot the mean over 5 runs. Imbalance $\approx 11.5\%$.

LAS-GNN and E-GIN show good resilience to changing thresholds and perform significantly better when r is low.

We see that integrating edge weight multiplication with signed message passing alone does not match the performance of E-GIN with weights as edge features. However, the latter’s performance can be matched by utilizing the LSTM aggregators. Since the only difference between the basic GNN with edge weight multiplication and Random LAS-GNN is the randomized LSTM aggregator, we can attribute the increase in performance entirely to the additional expressiveness provided by the LSTMs. Even though there is no temporal information to consider and the sequential order of neighbour embeddings is arbitrary, LSTM aggregation proves to be beneficial. Instead of LSTMs, the E-GIN model relies on MLPs in the graph neural network to detect the weighted patterns. The fact that MLPs are not bothered with learning from a particular sequence of inputs, which can form a distraction to Random LAS-GNN, can explain the slightly higher average precision scores of E-GIN. However, the difference is insignificant and we conclude that Random LAS-GNN is on par with the graph isomorphism network regarding the weighted non-temporal task.

The observations are similar for the gather-scatter and C3 patterns, only to a lesser extent due to the baseline GNN performance being better than for the scatter-gather pattern. The results of the three models on these patterns can be found in [Appendix A](#).

6.3.1 Edge Weight Similarity Task

In addition to the regular weighted task described in Chapter 5, we apply the same three models to a slightly different task. In this task, rather than considering high weights as suspicious, we focus on the similarity of weights within a distinct pattern. As seen in literature in Chapter 2, a money laundering pattern can also be deemed suspicious if the transactions within the pattern transfer similar amounts of money, resulting in a close-to-zero balance for the intermediate accounts. To test whether the models can cope with this scenario, we generate a mean value μ for each distinct pattern and assign the edges within the pattern a weight from a normal distribution centred around μ with a small standard deviation of $\sigma = 0.02$. Again, we modify these weights for approximately half of the patterns, while all other patterns are assigned edge weights from $U(0, 1)$.

The effectiveness of the models varies per pattern, as shown in Figure 6.6. We observe that all three models can easily detect gather-scatter patterns. However, for the scatter-gather pattern, none of the models achieve a satisfactory F1 score, and they do not seem confident in discerning the suspiciously weighted patterns from the randomly weighted patterns. While the same holds for the GNN with edge weight multiplication and Random LAS-GNN on the C3 task, we can see that the E-GIN model is effective here.

The results indicate that the edge similarity task is more challenging to learn than the regular weighted task, where suspicious weights are generally higher. A possible explanation is that for the latter, the suspicious weights are consistently higher across distinct patterns (weights are above r in each suspicious pattern). In contrast, there is no correlation between the weights across different patterns in the edge similarity task. Although none of the three models successfully captures suspicious weight similarity across all patterns, using edge features to model the weights appears more effective than edge weight multiplication. However, further analysis is required to determine if the advantage of edge features persists in scenarios with numerous other edge features. Additionally, we did not explore the combination of LSTM aggregation with co-embedded edge features, which could be beneficial.

Since this task is not successful on all three patterns for any of the trained models, we will stick to the weight definition described in Section 5.1.3 for the weighted temporal tasks.

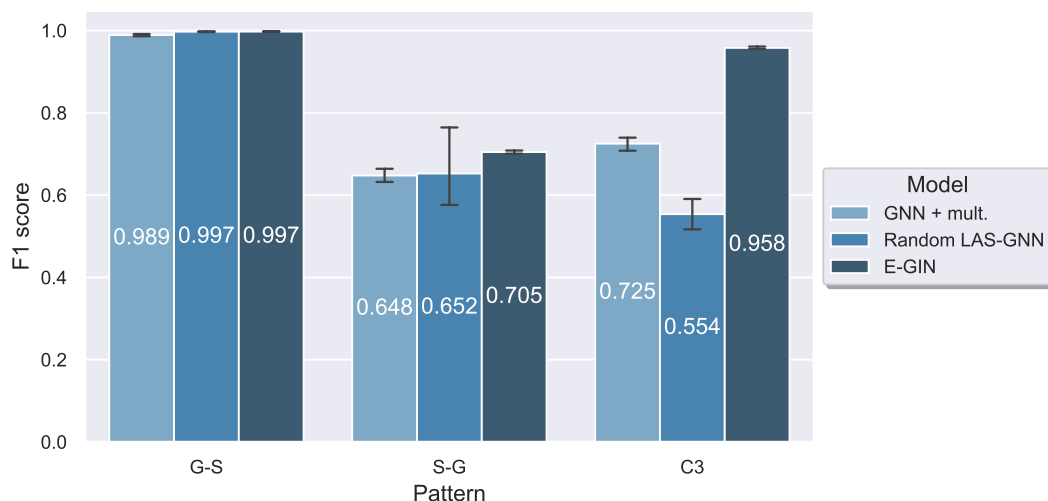


Figure 6.6: F1 scores on the edge weight similarity task. For each suspicious pattern, we generate the weights using a normal distribution with $\mu \in U(0, 1)$ and $\sigma = 0.02$. This task is balanced: we sampled 2000 weighted pattern nodes and an equal number of non-weighted pattern nodes at random from a 30K network.

6.4 Unweighted Temporal Detection

In this section, we discard the edge weights (i.e. setting each edge weight to 1) and instead focus on including edge timestamps. The GNN technique aimed at modelling this temporal aspect of the ML patterns is LSTM aggregation, as described in [Section 4.4](#). The LAS-GNN model utilizes LSTMs to aggregate neighbouring embeddings of a node in the order of their corresponding timestamps, allowing it to learn from sequential data. Alternatively, E-GIN is extended with the normalized timestamps as edge features, which are co-embedded with the node features and fed into MLPs. We compare these two models and provide two additional baselines: The Random and Hetero variants of LAS-GNN. The Random model is essentially a randomized LAS-GNN that does include LSTM aggregation but ignores the edge ordering specified by the timestamps. The Hetero model does maintain the sequential ordering of the edge timestamps but performs heterogeneous message passing instead of signed message passing. This means that for a node, the two different edge types (incoming and outgoing edges) are aggregated separately by distinct LSTMs. We compare the four approaches on generated networks with 10K nodes, where the timestamps are randomly assigned to edges and temporal patterns naturally occur. The precision-recall curves are shown in [Figure 6.7](#).

The PR curves clearly show that LAS-GNN can almost perfectly detect the temporal patterns,

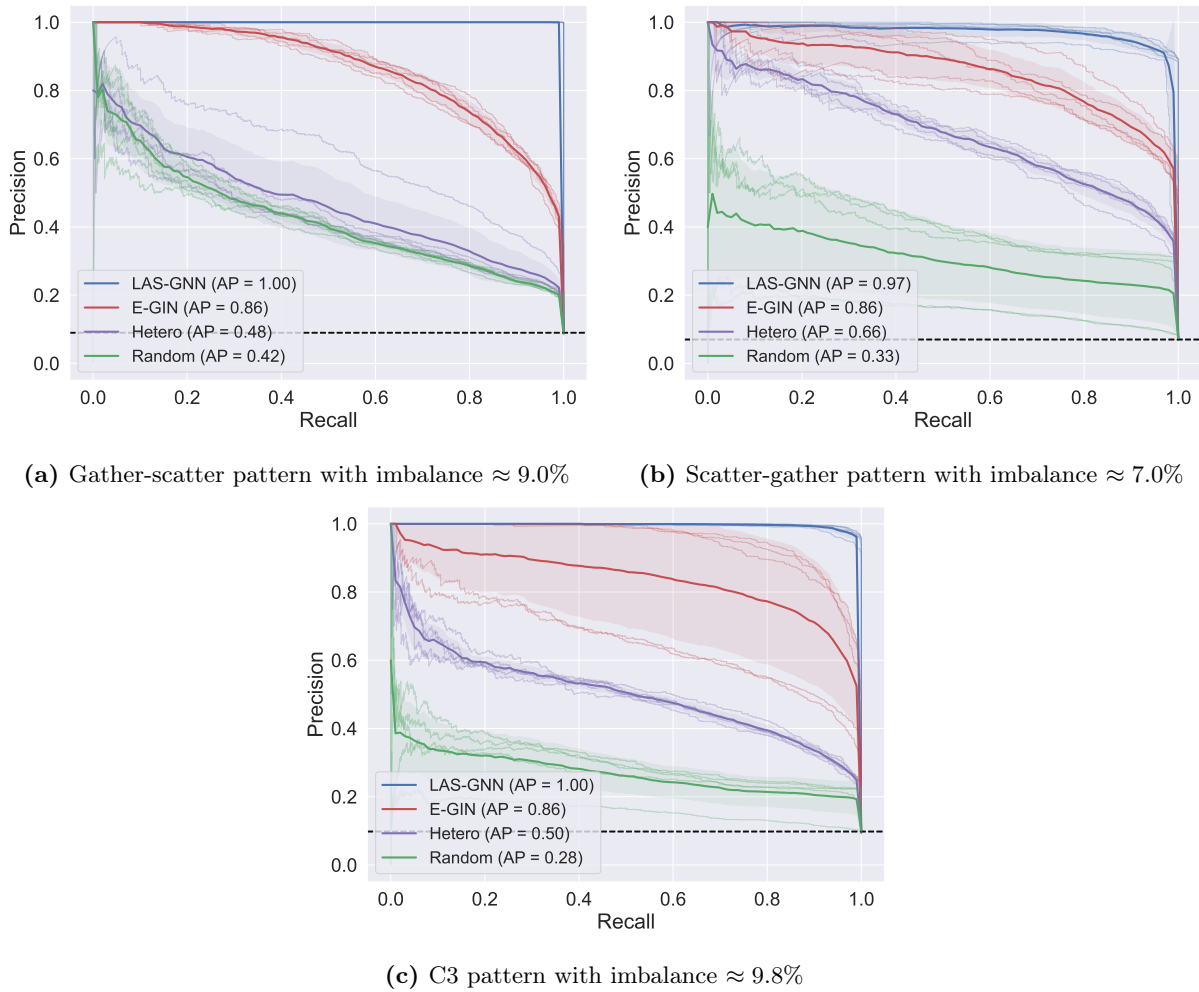


Figure 6.7: Precision-Recall curves for temporal pattern detection on networks with 10K nodes. Each plot is the mean over 5 runs. The filled area marks the region within a standard deviation from the mean PR curve.

while the Random and Hetero variants struggle to learn this temporal aspect. Although these two models are equipped with the necessary techniques to detect structural ML patterns, they both fail to achieve a decent average precision score for the less than 10% of nodes that belong to a suspicious temporal pattern. The failure of Random LAS-GNN is inevitable: by using a randomized LSTM, it does not incorporate temporal information in any way. Nevertheless, it serves as a reference point on this imbalanced task and supports the claim that LSTM aggregation only works as intended in temporal tasks when the edge ordering is respected. Even though the Hetero variant does make use of the temporal information by sorting the neighbour sequence by timestamp, its lacking performance demonstrates that separate aggregation, as proposed by heterogeneous message passing, is incompatible with learning the sequential ordering necessary to detect these temporal patterns.

E-GIN does manage to learn from the temporal pattern examples provided during training. However, with an average precision score of 0.86 for all three patterns, it falls short compared to LAS-GNN. Besides achieving a lower AP score, the PR curve shows that the performance can be inconsistent across multiple runs, particularly for the C3 pattern. This inconsistency is also evident in the weighted temporal detection tasks discussed in the next section, where we compare these two models more extensively.

6.5 Weighted Temporal Detection

After investigating the detection of weighted and temporal patterns separately in the previous two sections, we now address the combined task of identifying money laundering patterns that are both temporally ordered and weighted suspiciously. Since this task is more specific, the class imbalance is more severe. We train, validate, and test on networks with 100K nodes, using a batch size of 512 to accommodate. After randomly assigning timestamps to the edges, we extract all temporally ordered patterns using an exhaustive search. We then suspiciously weigh around half of these patterns using a weight threshold of $r = 0.8$. If we stop there, the combined task could be solved approximately by ignoring the timestamps and solely focusing on the edge weights. To ensure a fair task, we also set the weights of an equal number of non-temporally ordered patterns to be suspicious. Only the key nodes of temporally-ordered, suspiciously-weighted patterns are positively labeled. In subsequent subsections, we present the results of LAS-GNN compared to the E-GIN model for gather-scatter, scatter-gather, and C3 patterns separately.

Gather-scatter Pattern

The precision-recall curves of LAS-GNN and E-GIN are shown in [Figure 6.8a](#). As with the isolated weighted and temporal detection tasks, LAS-GNN achieves a nearly perfect average precision score in detecting gather-scatter patterns. The E-GIN model is not far behind and achieves a score of around 0.95 across all five runs. However, examining the training and validation loss progression in a single run, depicted in [Figure 6.8b](#), reveals that it takes the E-GIN model significantly more training rounds to learn the nuances of the weighted temporal patterns. Whereas LAS-GNN converges after just 15 epochs, the E-GIN model’s training loss only starts to decrease visibly after 40 epochs. Additionally, the validation loss of the E-GIN model exhibits considerable instability from that point onward. Although small spikes in training and validation loss are common in mini-batch training, the pronounced oscillations in the E-GIN model’s validation loss indicate unstable generalization. This even leads to early convergence due to the validation loss not improving over the last 20 epochs, something that is evident in four out of the five runs. The loss progress of the other runs can be found in [Appendix B](#).

To provide more insights, we report confusion matrices for both LAS-GNN and E-GIN in [Table 6.3](#). Since a confusion matrix corresponds to a specific classification threshold, with a fixed balance between precision and recall, we select the E-GIN model’s result at its best classification

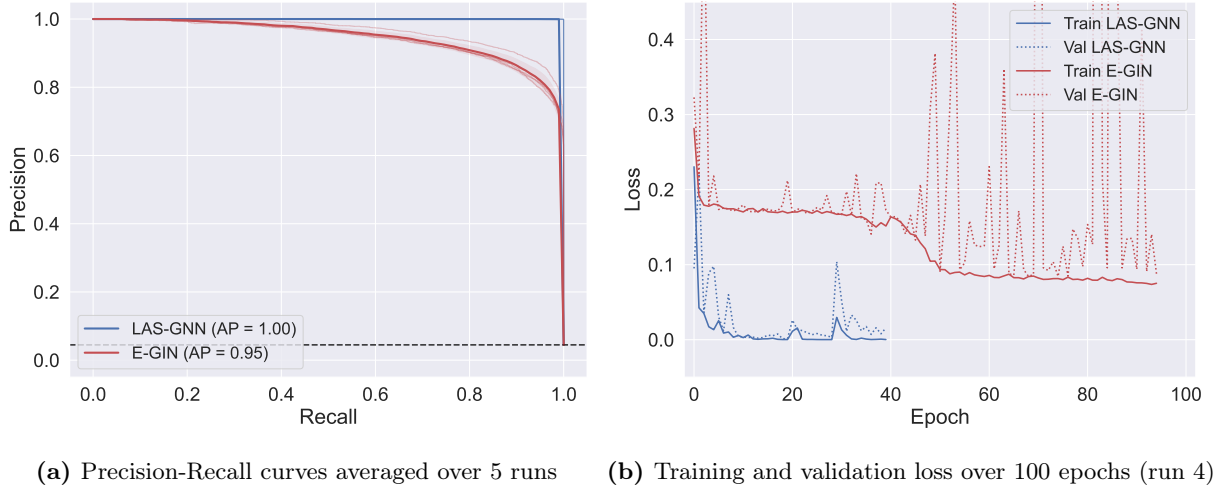


Figure 6.8: Results of gather-scatter pattern detection on weighted temporal networks of 100K nodes. The imbalance is 4.5%.

		Predicted	
		1	0
Actual	1	4494	0
	0	4	95502

(a) LAS-GNN with $F1 = 1.000$, $P = 0.999$ and $R = 1.000$

		Predicted	
		1	0
Actual	1	4247	247
	0	616	94890

(b) E-GIN with $F1 = 0.908$, $P = 0.873$ and $R = 0.945$

Table 6.3: Confusion matrices of the models at their best classification threshold, measured by $F1$ score. Corresponding precision and recall values are provided.

threshold across all five runs, as measured by the $F1$ score. For comparison, we picked the best result of LAS-GNN on the same network as the E-GIN model. We see that LAS-GNN accurately identifies all 4494 suspicious weighted temporal patterns, with only four non-suspicious nodes misclassified as key nodes. In contrast, the E-GIN model produces 616 false positives and 247 false negatives, resulting in its best $F1$ score of 0.908.

Scatter-gather Pattern

For the scatter-gather pattern, LAS-GNN achieves excellent performance with an average precision score of 0.98. From Figure 6.9b, it is clear that E-GIN does not suffer as badly from oscillations in the validation loss as it does for the gather-scatter pattern. However, once the loss is just below 0.2, it decreases slowly, nearly appearing converged but not quite reaching full convergence. The same progression occurs in all five runs (see Appendix B). According to the PR curves of the E-GIN model in Figure 6.9a, its average performance is negatively impacted by a single poor run. However, even in its best-performing run, it can not match the performance of LAS-GNN.

When we look at the best confusion matrix of E-GIN in Table 6.4, we see that it has thrice the amount of false positives, almost double the number of false negatives, and its $F1$ score is around 0.1 lower, compared to LAS-GNN. In the case of this specific classification threshold, this can mostly be attributed to the lacking precision score.

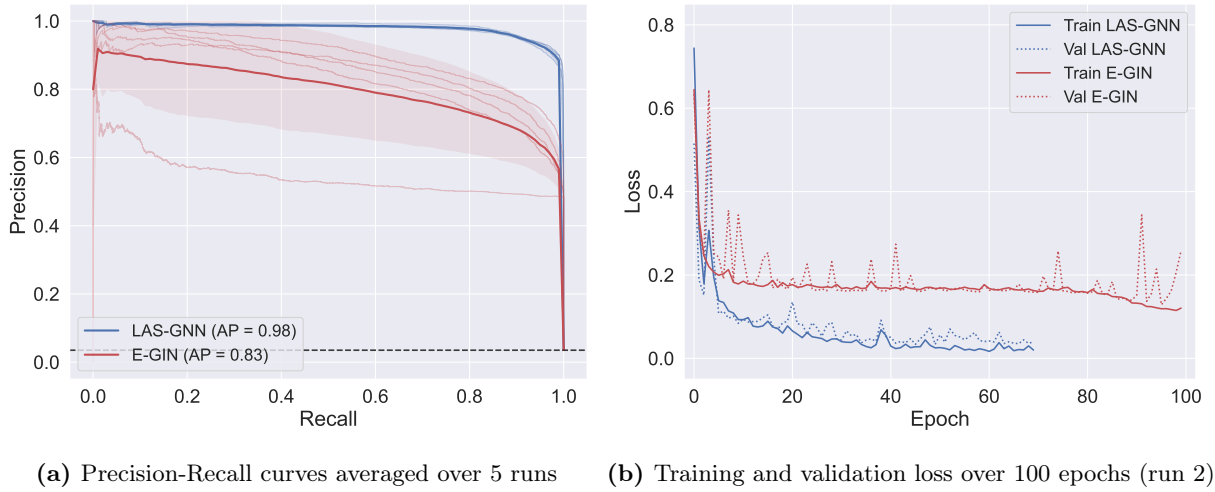


Figure 6.9: Results of scatter-gather pattern detection on weighted temporal networks of 100K nodes. The imbalance is 3.5%.

		Predicted	
		1	0
Actual	1	3394	171
	0	306	96129

(a) LAS-GNN with $F1 = 0.934$, $P = 0.917$ and $R = 0.952$

		Predicted	
		1	0
Actual	1	3256	309
	0	943	95492

(b) E-GIN with $F1 = 0.839$, $P = 0.775$ and $R = 0.913$

Table 6.4: Confusion matrices of the models at their best classification threshold, measured by $F1$ score. Corresponding precision and recall values are provided.

C3 Pattern

For the C3 pattern, the LAS-GNN model achieves outstanding performance as well, see [Figure 6.10](#) and [Table 6.5](#). Its loss curve declines the most rapidly among all three patterns. The model converges within a few epochs, although it does not attain the perfect (rounded) average precision score achieved with the gather-scatter pattern.

E-GIN appears to suffer from both issues we observed in the gather-scatter and scatter-gather detection tasks. Firstly, its validation loss exhibits significant spikes. Despite this, the overall trend of the validation loss is downward, suggesting that it might eventually match the performance of LAS-GNN with additional training rounds. However, in 3 out of 5 runs, this does not materialize and the loss plateaus (See [Appendix B](#)). Similar to the gather-scatter pattern, there is also a plateau in the first 30 epochs. Secondly, the precision-recall curves of E-GIN in [Figure 6.10a](#) are not as consistent as for the gather-scatter pattern, with variability between the different runs. This indicates that while the E-GIN model is able to achieve satisfactory performance, it may not be reliable. Nonetheless, we do not observe an outlier as pronounced as the one seen in [Figure 6.9a](#) for the scatter-gather pattern.

Looking at the results of both LAS-GNN and E-GIN on all three patterns, we can conclude that the gather-scatter patterns is the easiest pattern to detect out of these three, followed by the C3 pattern. The scatter-gather pattern seems to be the most challenging pattern to correctly identify.

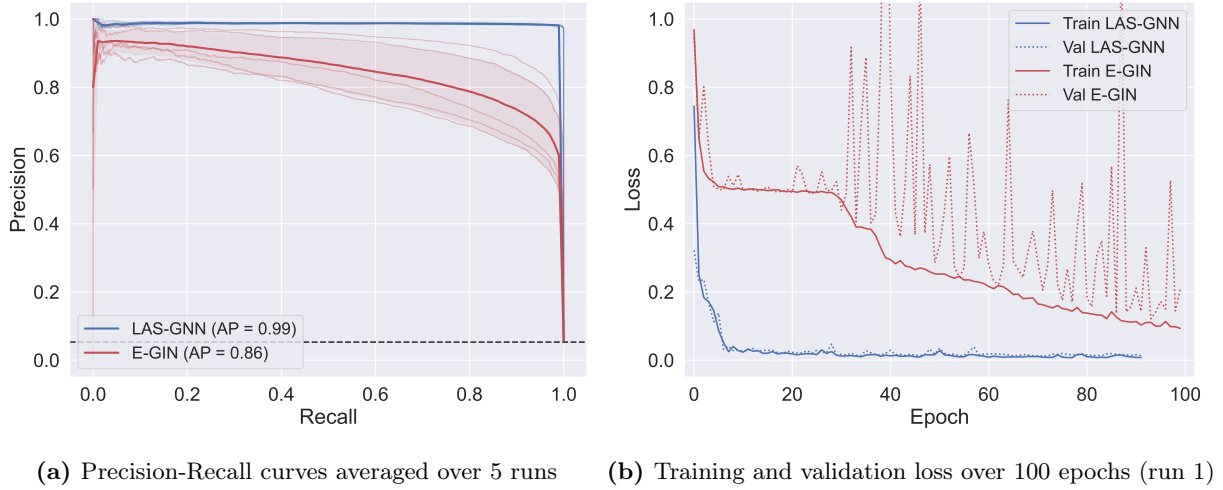


Figure 6.10: Results of C3 pattern detection on weighted temporal networks of 100K nodes. The imbalance is 5.1%.

		Predicted	
		1	0
Actual	1	5707	17
	0	133	94143

(a) LAS-GNN with $F1 = 0.987$, $P = 0.977$ and $R = 0.997$

		Predicted	
		1	0
Actual	1	5377	347
	0	916	93360

(b) E-GIN with $F1 = 0.895$, $P = 0.854$ and $R = 0.939$

Table 6.5: Confusion matrices of the models at their best classification threshold, measured by $F1$ score. Corresponding precision and recall values are provided.

6.6 Longer Cycle Detection

Finally, we investigate simple cycle patterns of length 4, 5, and 6. We specifically focus on the temporal task here since we want to test the LSTM aggregator’s effectiveness for longer cycles beyond the extensively studied C3 pattern. This task is balanced, with 2000 temporal cycles and 2000 non-temporal cycles sampled at random. To generate longer temporal cycles, we need to create large networks, as the probability of a long cycle being temporally ordered after a random assignment of edge timestamps is very low. For C3, C4, C5 and C6, we generate networks with 30K, 100K, 500K, and 4 million nodes, respectively. With these settings, we can expect at least 2000 temporally ordered patterns to occur in the data. We train, validate, and test LAS-GNN and E-GIN on distinct synthetic networks of the specified size. To detect a cycle of length n , we use n LSTM layers following a single ADD layer in LAS-GNN, while we use $n + 1$ ADD layers for the E-GIN model. The average $F1$ scores achieved by the MPNN models are presented in Figure 6.11.

As expected, the average $F1$ score of the detection models decreases as the cycle length increases. Nevertheless, LAS-GNN only shows a drop of 0.039 from C3 to C5 patterns, while E-GIN, which already had a lower $F1$ score of 0.928 for C3, declines with 0.203. Furthermore, LAS-GNN is consistent across multiple runs, as indicated by the small standard deviation represented by the error bars. On the other hand, the error bars of E-GIN show variability in the outcomes, which we also observed for other ML patterns in previous experiments. The difference

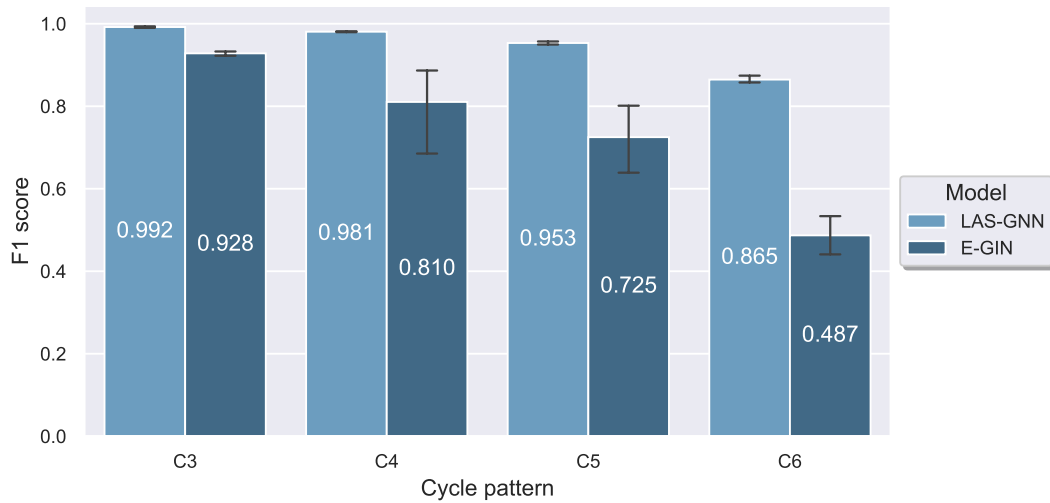


Figure 6.11: F1 scores for temporal pattern detection of longer cycles. Results are averaged across 5 runs. This task is balanced: we sampled 2000 temporal cycle patterns and 2000 non-temporal cycle patterns at random.

in performance between the two models grows with each longer cycle. The most notable difference between the two GNNs is for C6 patterns: LAS-GNN still achieves a commendable F1 score of 0.865, whereas E-GIN fails to effectively discern the temporal cycles from the non-temporal cycles and achieves an F1 score of 0.487.

We can conclude that having the timestamps as edge features is much less robust against longer cycles than utilizing LSTM aggregation as done in LAS-GNN. While we only tested on longer cycles, this could mean that the latter is a more promising approach for larger subgraph patterns that require message passing at further depth.

Chapter 7

Discussion

7.1 Discussion of Results

In the preliminary experiments in [Section 6.1.1](#), we reproduced the results of [Egressy et al. \(2024\)](#), demonstrating that heterogeneous message passing combined with the addition of ego IDs to the node features is essential to correctly identify the basic money laundering patterns. Furthermore, our alternative approach of signed message passing seems to be just as effective, achieving perfect detection results in directed graphs with reciprocating edges. Furthermore, we observed that at least four GNN layers are required to detect all basic ML patterns, while for temporal pattern detection two LSTM layers can be sufficient, although ADD aggregation in the first layer can stabilize performance and speed up convergence.

When including edge weights in [Section 6.3](#), we saw that the performance of a GNN integrating edge weight multiplication with signed message passing can be enhanced by incorporating LSTM aggregation, even when there is no temporal information to consider. The performance is then on par with the approach of setting the weights as edge features, as done for the E-GIN model. However, the weight similarity task of [Section 6.3.1](#) was unsuccessful and more investigation is necessary into the advantages of weight multiplication over features under other conditions.

For the temporal tasks, we observed in [Section 6.4](#) that standard LAS-GNN clearly outperforms its Random and Heterogeneous variants. This suggested that LSTM aggregation only works as intended when the edge ordering is respected and the message passing allows for simultaneous aggregation of incoming and outgoing neighbour embeddings. In contrast to heterogeneous message passing, our novel signed message passing does enable an LSTM to learn the dependencies between the two directions.

The LAS-GNN model also outperforms the E-GIN benchmark, in both the isolated temporal task and the combined task with edge weights included. Although the E-GIN model can be effective, it never achieves the near-perfect performance of LAS-GNN. This is not the case for the weights-only task, indicating that the difference between the two models is primarily caused by the temporal aspect. Further investigation into the loss progression in [Section 6.5](#) shows that the E-GIN model requires many training rounds to learn the nuances of the temporal characteristics. Its lack of stability, evident by heavy oscillations in the loss, and slow convergence sometimes prevent it from achieving satisfactory results, and even when it does, it takes many epochs. This suggests that, even though all information is available through the edge features, E-GIN struggles to learn from this information effectively. E-GIN must independently determine that the timestamps represent a sequential order, whereas LAS-GNN explicitly models this in its aggregation mechanism. Even when data imbalance is not a concern, and we focus solely on distinguishing temporal cycles from non-temporal cycles in a balanced task, having the timestamps as edge features seems insufficient. While some performance drop for longer cycles is expected, as the model relies on embeddings from further away, the E-GIN's decline in performance is significant.

The contrast with LAS-GNN, which requires significantly fewer training rounds, shows consistency across multiple runs, and is more robust to longer cycle patterns, further supports

the effectiveness of the proposed LSTM aggregation combined with signed message passing in learning from temporal information in dynamic networks. Besides its superior performance over the alternative of including time through straightforward features, it is also more elegant than using time-based graph slices. Moreover, LAS-GNN integrates the temporal feature extraction of the LSTM seamlessly within the graph neural network, instead of this being a smaller component separate from the spatial feature extraction of the GNN within a larger, more complex model, as the ones discussed in [Chapter 2](#).

7.2 Experimental Limitations

The experiments do not showcase the full potential of LAS-GNN. Our work does not include the GNN technique of port numbering for identifying patterns in directed multi-graphs, and this aspect remains to be tested. Moreover, the model should be evaluated on other types of networks as well. As mentioned in [Section 5.1](#), the Watts-Strogatz network generation model produces an unrealistic degree distribution. Therefore, it is essential to investigate how LAS-GNN adapts to a power-law degree distribution when applying the model to real-world networks that contain hubs.

Another direction for future research is to apply the GNN techniques to a real-world dataset, including a range of node and edge features. Our work focused on controlled tasks, using only the features necessary for the GNN techniques. However, a major advantage of neural networks is their ability to extract relevant features from raw data and learn complex relationships among numerous input features. Such features could also include pre-calculated features based on a balancing score or certain centrality measures, as the ones described in [Chapter 2](#).

7.3 Model Limitations

When we include a large set of features, it remains to be seen how this affects LAS-GNN. Over-smoothing is an issue that causes information loss in the embeddings after many rounds of message passing. Since a large number of GNN layers may be required for larger patterns, such as long cycles, it can potentially become difficult to distinguish embeddings derived from many features. Moreover, since the ego ID is essentially modeled as a node feature, it risks being overshadowed by the abundance of other features, negatively impacting the ability of the model to detect cycles in which the node participates. In contrast, this issue would not arise for edge weights in case we utilize edge weight multiplication, which could offer an advantage over treating edge weights as edge features, as is done for the E-GIN model.

Furthermore, the approach of conducting node-level classification to identify the key nodes in suspicious patterns does not extract the relevant patterns from the data. Future work can focus on developing methods that return complete patterns using the results of the node classification. Especially for patterns that contain more than one key node, such as bipartite-clique and stack patterns, it would be beneficial to combine the related positively predicted nodes to form the actual pattern. For instance, one could employ similarity metrics or proximity-based random walk embeddings to find nodes with a high probability of belonging to the same pattern. For the node classification itself, it remains to be seen how the model performs when we aim to detect different types of patterns simultaneously within the binary classification setting (suspicious or not). Utilizing an ensemble of GNNs might offer a solution, although a first step could be to adapt our model to the multi-class setting, where each topology has a distinct label to predict.

After addressing potential shortcomings, other applications can be explored. While we applied our model to money laundering detection specifically, our findings can be generalized to other domains that include similar patterns, such as molecule classification and traffic forecasting.

Chapter 8

Conclusion

In this research, we addressed the problem of detecting suspicious accounts involved in money laundering using machine learning, which can overcome many of the limitations of traditional rule- and scenario-based methods still used at banks today. While numerous anomaly detection and supervised machine learning approaches operate directly on transaction data, we focused on deep learning methods that incorporate the underlying network structure representing relationships in such data. Among techniques such as graph-based features, random walk embeddings, and graph neural networks, the latter seems most promising for detecting money laundering patterns.

However, we saw that GNNs alone are ill-equipped to identify suspicious subgraph patterns in financial networks. Although [Egressy et al. \(2024\)](#) proposed the techniques of ego IDs and heterogeneous message passing to enhance GNNs for this task, they do not capture the dynamics of temporal networks. Approaches incorporating temporal information include straightforward techniques like sliding time windows and temporal features. Overly complex models integrate recurrent neural networks, such as LSTMs, within a larger framework, where the temporal feature extraction is often separate from the GNN conducting spatial feature extraction while still relying on time-based graph slices to generate inputs for the LSTM.

Instead, we combined the GNN techniques proposed by [Egressy et al. \(2024\)](#) with a novel way of incorporating the weighted and temporal properties of money laundering patterns seamlessly integrated within a single GNN. We embedded an LSTM-based aggregation mechanism in the GNN layers to learn from the sequential order imposed by edge timestamps. We proposed signed message passing as an alternative to heterogeneous message passing to enable simultaneous aggregation of incoming and outgoing messages while preserving edge directionality. Additionally, we incorporated edge weight multiplication for dealing with weighted patterns.

Near-perfect detection results demonstrate that our LAS-GNN model is well-suited for structural pattern-finding tasks, including those with edge weights and timestamps. Key nodes involved in basic money laundering patterns can be classified flawlessly within synthetically generated directed Watts-Strogatz networks with reciprocating edges. For gather-scatter, scatter-gather and C3 patterns, the model achieved excellent average precision scores on weighted and temporal networks, even under severe class imbalance. Compared to a GIN benchmark model with weights and timestamps incorporated as edge features, LAS-GNN requires significantly fewer training rounds and shows consistency across multiple runs. These impressive results also extend to longer cycle patterns, although there is a slight decline in performance.

Further investigation is necessary to adapt the model to real-world datasets that consist of more realistic networks and numerous features. Beyond its application to money laundering pattern detection, our findings can then be generalized to other domains involving weighted temporal networks.

Bibliography

- R. Abboud, I. I. Ceylan, M. Grohe, and T. Lukasiewicz. The surprising power of graph neural networks with random node initialization. *arXiv preprint arXiv:2010.01179*, 2020.
- A. O. Adewumi and A. A. Akinyelu. A survey of machine-learning and nature-inspired based credit card fraud detection techniques. *International Journal of System Assurance Engineering and Management*, 8:937–953, 2017.
- L. Akoglu, M. McGlohon, and C. Faloutsos. Oddball: Spotting anomalies in weighted graphs. In *Advances in Knowledge Discovery and Data Mining: 14th Pacific-Asia Conference, PAKDD 2010*, pages 410–421. Springer, 2010.
- I. Alarab and S. Prakoonwit. Graph-based lstm for anti-money laundering: Experimenting temporal graph convolutional network with bitcoin data. *Neural Processing Letters*, 55(1): 689–707, 2023.
- I. Alarab, S. Prakoonwit, and M. I. Nacer. Comparative analysis using supervised learning methods for anti-money laundering in bitcoin. In *Proceedings of the 2020 5th international conference on machine learning technologies*, pages 11–17, 2020.
- A. Ali, S. Abd Razak, S. H. Othman, T. A. E. Eisa, A. Al-Dhaqm, M. Nasser, T. Elhassan, H. Elshafie, and A. Saif. Financial fraud detection based on machine learning: a systematic literature review. *Applied Sciences*, 12(19):9637, 2022.
- E. Altman, J. Blanuša, L. Von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu. Realistic synthetic financial transactions for anti-money laundering models. *Advances in Neural Information Processing Systems*, 36, 2024.
- N. Bakhshinejad. A graph-based deep learning model for anti-money laundering. 2023. Master’s thesis, York University (Canada).
- Capgemini. The growing threat of money laundering: The significant role financial services institutions can play in curbing money laundering activities. URL https://www.capgemini.com/wp-content/uploads/2017/07/The_Growing_Threat_of_Money_Laundering.pdf.
- M. Cardoso, P. Saleiro, and P. Bizarro. Laundrograph: Self-supervised graph representation learning for anti-money laundering. In *Proceedings of the Third ACM International Conference on AI in Finance*, pages 130–138, 2022.
- Z. Chen, L. D. Van Khoa, E. N. Teoh, A. Nazir, E. K. Karuppiah, and K. S. Lam. Machine learning techniques for anti-money laundering (aml) solutions in suspicious transaction detection: a review. *Knowledge and Information Systems*, 57:245–285, 2018.
- A. F. Colladon and E. Remondi. Using social network analysis to prevent money laundering. *Expert Systems with Applications*, 67:49–58, 2017.
- S. Das, W.-K. Wong, T. Dietterich, A. Fern, and A. Emmott. Incorporating expert feedback into active anomaly discovery. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*, pages 853–858. IEEE, 2016.

- J. Davis and M. Goadrich. The relationship between precision-recall and roc curves. In *Proceedings of the 23rd international conference on Machine learning*, pages 233–240, 2006.
- De Nederlandse Bank. Post-event transactiemonitoringsproces bij banken. URL <https://www.dnb.nl/media/0eilgixn/guidance-document-transactiemonitoring-banken.pdf>.
- R. Dreżewski, J. Sepielak, and W. Filipkowski. The application of social network analysis algorithms in a system supporting money laundering detection. *Information Sciences*, 295: 18–32, 2015.
- B. Dumitrescu, A. Băltoiu, and Ş. Budulan. Anomaly detection in graphs of bank transactions for anti money laundering applications. *IEEE Access*, 10:47699–47714, 2022.
- B. Egressy and R. Wattenhofer. Graph neural networks with precomputed node features. *arXiv preprint arXiv:2206.00637*, 2022.
- B. Egressy, L. Von Niederhäusern, J. Blanuša, E. Altman, R. Wattenhofer, and K. Atasu. Provably powerful graph neural networks for directed multigraphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 11838–11846, 2024.
- M. Fey and J. E. Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- S. Gao, D. Xu, H. Wang, and P. Green. Knowledge-based anti-money laundering: a software agent bank application. *Journal of Knowledge Management*, 13(2):63–75, 2009.
- M. Gori, G. Monfardini, and F. Scarselli. A new model for learning in graph domains. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 2, pages 729–734. IEEE, 2005.
- O. M. Granados and A. Vargas. The geometry of suspicious money laundering activities in financial networks. *EPJ Data Science*, 11(1):6, 2022.
- A. Grover and J. Leskovec. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864, 2016.
- A. Hagberg, P. Swart, and D. S Chult. Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2008.
- W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.
- H. He and E. A. Garcia. Learning from imbalanced data. *IEEE Transactions on knowledge and data engineering*, 21(9):1263–1284, 2009.
- J. He, J. Tian, Y. Wu, X. Cia, K. Zhang, M. Guo, H. Zheng, J. Wu, and Y. Ji. An efficient solution to detect common topologies in money launderings based on coupling and connection. *IEEE Intelligent Systems*, 36(1):64–74, 2021.
- S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos. Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 895–904, 2016.

- W. Hu, B. Liu, J. Gomes, M. Zitnik, P. Liang, V. Pande, and J. Leskovec. Strategies for pre-training graph neural networks. In *International Conference on Learning Representations*, 2020.
- S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- M. Jullum, A. Løland, R. B. Huseby, G. Ånonsen, and J. Lorentzen. Detecting money laundering transactions with machine learning. *Journal of Money Laundering Control*, 23(1):173–186, 2020.
- B. Khemani, S. Patil, K. Kotecha, and S. Tanwar. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. *Journal of Big Data*, 11(1):18, 2024.
- D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017.
- D. Labanca, L. Primerano, M. Markland-Montgomery, M. Polino, M. Carminati, and S. Zanero. Amaretto: an active learning framework for money laundering detection. *IEEE Access*, 10: 41720–41739, 2022.
- N. M. Labib, M. A. Rizka, and A. E. M. Shokry. Survey of machine learning approaches of anti-money laundering techniques to counter terrorism finance. In *Internet of Things—Applications and future: Proceedings of ITAF 2019*, pages 73–87. Springer.
- A. Lehman and B. Weisfeiler. A reduction of a graph to a canonical form and an algebra arising during this reduction. *Nauchno-Technicheskaya Informatsiya*, 2(9):12–16, 1968.
- X. Li, S. Liu, Z. Li, X. Han, C. Shi, B. Hooi, H. Huang, and X. Cheng. Flowscope: Spotting money laundering based on graphs. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 4731–4738, 2020.
- X. Li, Y. Li, X. Mo, H. Xiao, Y. Shen, and L. Chen. Diga: Guided diffusion model for graph recovery in anti-money laundering. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4404–4413, 2023.
- S. Liu, B. Hooi, and C. Faloutsos. A contrast metric for fraud detection in rich graphs. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2235–2248, 2018.
- E. Lopez-Rojas, A. Elmir, and S. Axelsson. Paysim: A financial mobile money simulator for fraud detection. In *28th European Modeling and Simulation Symposium, EMSS, Larnaca*, pages 249–255. Dime University of Genoa, 2016.
- J. S. Lorenz. *Machine learning methods to detect money laundering in the Bitcoin blockchain in the presence of label scarcity*. PhD thesis, Universidade NOVA de Lisboa (Portugal), 2021.
- L. McInnes, J. Healy, and J. Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- M. McPherson, L. Smith-Lovin, and J. M. Cook. Birds of a feather: Homophily in social networks. *Annual review of sociology*, 27(1):415–444, 2001.

- P. Mishra, A. Piktus, G. Goossen, and F. Silvestri. Node masking: Making graph neural networks generalize and scale better. *arXiv preprint arXiv:2001.07524*, 2020.
- F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in python. *the Journal of machine Learning research*, 12:2825–2830, 2011.
- B. Perozzi, R. Al-Rfou, and S. Skiena. Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 701–710, 2014.
- L. F. Ribeiro, P. H. Saverese, and D. R. Figueiredo. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 385–394, 2017.
- B. V. Rompaye. Machine learning for anti-money laundering (ML for AML) - KPMG belgium. URL <https://kpmg.com/be/en/home/insights/2023/08/lh-machine-learning-for-anti-money-laundering.html>.
- E. Rossi, B. Charpentier, F. Di Giovanni, F. Frasca, S. Günnemann, and M. M. Bronstein. Edge directionality improves learning on heterophilic graphs. In *Learning on Graphs Conference*, pages 25–1. PMLR, 2024.
- R. Sato, M. Yamada, and H. Kashima. Approximation ratios of graph neural networks for combinatorial problems. *Advances in Neural Information Processing Systems*, 32, 2019.
- R. Sato, M. Yamada, and H. Kashima. Random features strengthen graph neural networks. In *Proceedings of the 2021 SIAM international conference on data mining (SDM)*, pages 333–341. SIAM, 2021.
- D. Savage, Q. Wang, P. Chou, X. Zhang, and X. Yu. Detection of money laundering groups using supervised learning in networks. *arXiv preprint arXiv:1608.00708*, 2016.
- A. Saxena, Y. Pei, J. Veldsink, W. van Ipenburg, G. Fletcher, and M. Pechenizkiy. The banking transactions dataset and its comparative analysis with scale-free networks. In *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, pages 283–296, 2021.
- F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.
- M. Schlichtkrull, T. N. Kipf, P. Bloem, R. Van Den Berg, I. Titov, and M. Welling. Modeling relational data with graph convolutional networks. In *The semantic web: 15th international conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, proceedings 15*, pages 593–607. Springer, 2018.
- B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson. Estimating the support of a high-dimensional distribution. *Neural computation*, 13(7):1443–1471, 2001.
- K. Shin, B. Hooi, J. Kim, and C. Faloutsos. D-cube: Dense-block detection in terabyte-scale tensors. In *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining*, pages 681–689, 2017.
- A. M. Shokry, M. A. Rizka, and N. M. Labib. Counter terrorism finance by detecting money laundering hidden networks using unsupervised machine learning algorithm. In *International Conferences ICT, Society, and Human Beings*, pages 89–97, 2020.

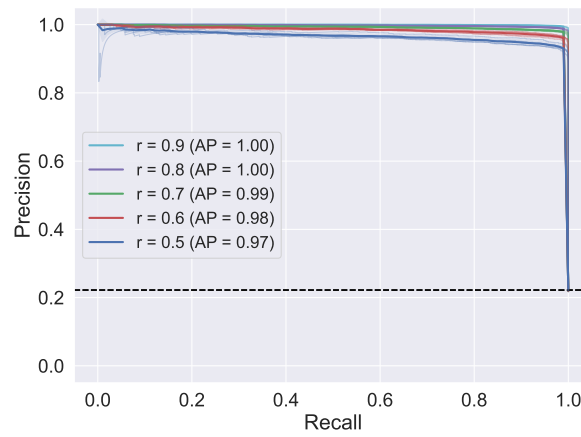
- R. Soltani, U. T. Nguyen, Y. Yang, M. Faghani, A. Yagoub, and A. An. A new algorithm for money laundering detection based on structural similarity. In *2016 IEEE 7th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pages 1–7. IEEE, 2016.
- Z. Song, X. Yang, Z. Xu, and I. King. Graph-based semi-supervised learning: A comprehensive review. *IEEE Transactions on Neural Networks and Learning Systems*, 2022.
- M. Starnini, C. E. Tsourakakis, M. Zamanipour, A. Panisson, W. Allasia, M. Fornasiero, L. L. Puma, V. Ricci, S. Ronchiadin, A. Ugrinoska, et al. Smurf-based anti-money laundering in time-evolving transaction networks. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part IV 21*, pages 171–186. Springer, 2021.
- X. Sun, W. Feng, S. Liu, Y. Xie, S. Bhatia, B. Hooi, W. Wang, and X. Cheng. Monlad: Money laundering agents detection in transaction streams. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*, pages 976–986, 2022.
- T. Suzumura and H. Kanezashi. Anti-Money Laundering Datasets: InPlusLab anti-money laundering datadatasets. <http://github.com/IBM/AMLSim/>, 2021.
- J. Tang and J. Yin. Developing an intelligent data discriminating system of anti-money laundering based on svm. In *2005 International conference on machine learning and cybernetics*, volume 6, pages 3453–3457. IEEE, 2005.
- UNODC. United nations office on drugs and crime. URL [//www.unodc.org/unodc/en/money-1aundering/overview.html](http://www.unodc.org/unodc/en/money-1aundering/overview.html).
- D. Wagner. Latent representations of transaction network graphs in continuous vector spaces as features for money laundering detection. *SKILL 2019-Studierendenkonferenz Informatik*, 2019.
- F. Wan and P. Li. A novel money laundering prediction model based on a dynamic graph convolutional neural network and long short-term memory. *Symmetry*, 16(3):378, 2024.
- S.-N. Wang and J.-G. Yang. A money laundering risk evaluation method based on decision tree. In *2007 international conference on machine learning and cybernetics*, volume 1, pages 283–286. IEEE, 2007.
- D. J. Watts and S. H. Strogatz. Collective dynamics of ‘small-world’ networks. *nature*, 393(6684):440–442, 1998.
- M. Weber, J. Chen, T. Suzumura, A. Pareja, T. Ma, H. Kanezashi, T. Kaler, C. E. Leiserson, and T. B. Schardl. Scalable graph learning for anti-money laundering: A first look. *arXiv preprint arXiv:1812.00076*, 2018.
- M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.
- P. Xia, Z. Ni, H. Xiao, X. Zhu, and P. Peng. A novel spatiotemporal prediction approach based on graph convolution neural networks and long short-term memory for money laundering fraud. *Arabian Journal for Science and Engineering*, 47(2):1921–1937, 2022.
- K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019.

- J. You, J. M. Gomes-Selman, R. Ying, and J. Leskovec. Identity-aware graph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 10737–10745, 2021.
- Q. Yuan, B. Huang, J. Zhang, J. Wu, H. Zhang, and X. Zhang. Detecting phishing scams on ethereum based on transaction records. In *2020 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5. IEEE, 2020.
- Zhang and P. Trubey. Machine learning and sampling scheme: An empirical study of money laundering detection. *Computational Economics*, 54:1043–1063, 2019.
- Zhang, J. J. Salerno, and P. S. Yu. Applying data mining in investigating money laundering crimes. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 747–752, 2003.
- A. Zhou, J. Yang, Y. Gao, T. Qiao, Y. Qi, X. Wang, Y. Chen, P. Dai, W. Zhao, and C. Hu. Brief industry paper: Optimizing memory efficiency of graph neural networks on edge computing platforms. In *2021 IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 445–448. IEEE, 2021.

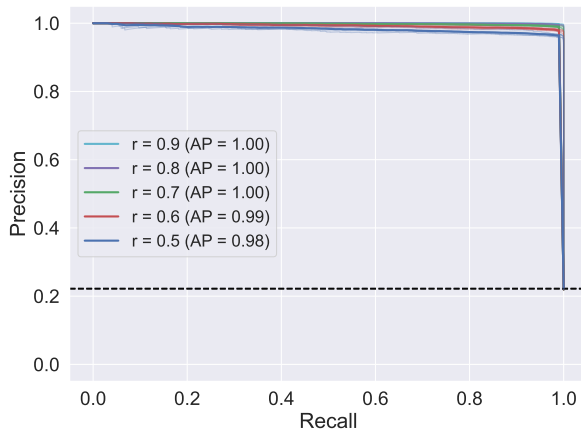
Appendix A

Weighted Non-temporal Detection

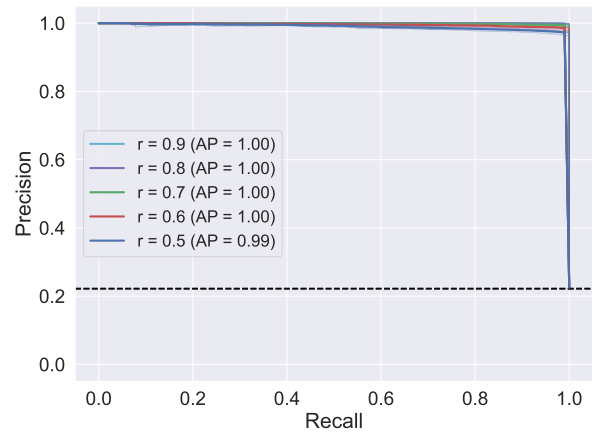
A.1 Gather-scatter



(a) GNN with edge weight multiplication



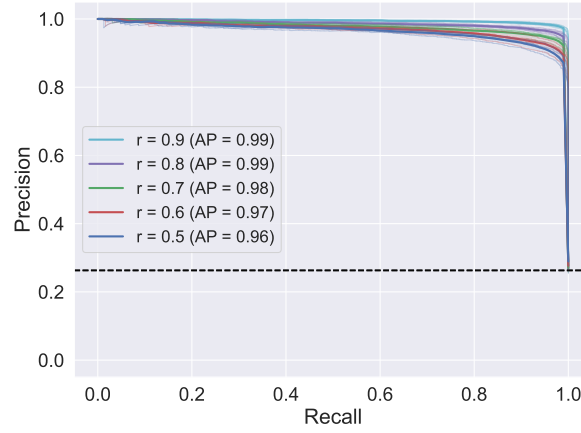
(b) Random LAS-GNN



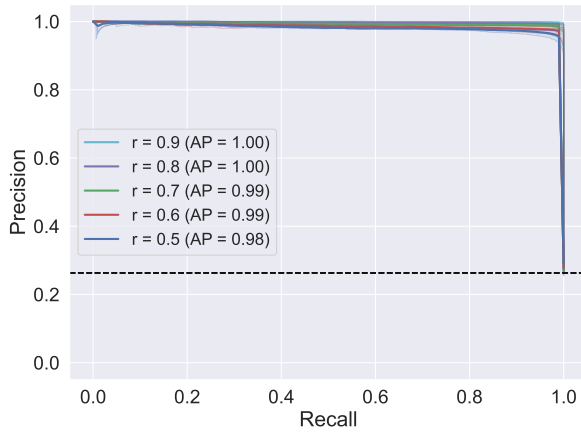
(c) E-GIN with edge weight as feature

Figure A.1: Precision-Recall curves for weighted gather-scatter pattern detection on networks with 10K nodes. For each weight threshold value r , we plot the mean over 5 runs. Imbalance $\approx 22.2\%$.

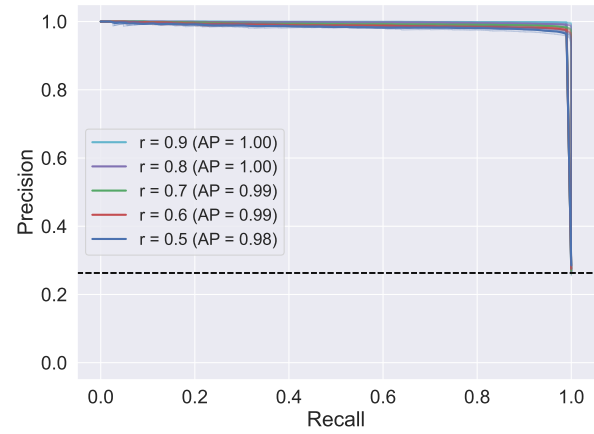
A.2 C3



(a) GNN with edge weight multiplication



(b) Random LAS-GNN



(c) E-GIN with edge weight as feature

Figure A.2: Precision-Recall curves for weighted C3 pattern detection on networks with 10K nodes. For each weight threshold value r , we plot the mean over 5 runs. Imbalance $\approx 26.3\%$.

Appendix B

Weighted Temporal Detection

On the following pages, we provide the loss progression plots of the models for each pattern across all five runs.

B.1 Gather-scatter

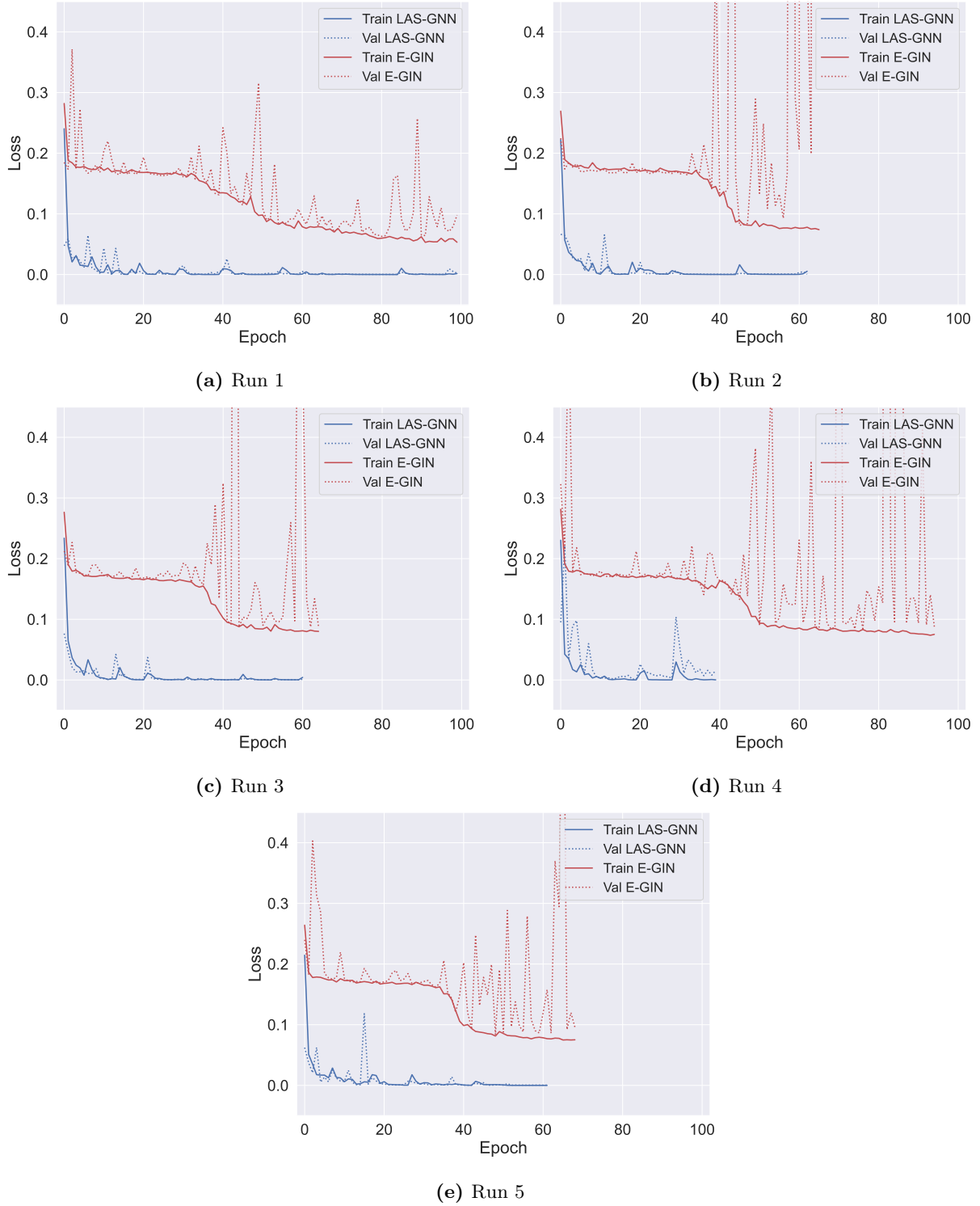


Figure B.1: Training and validation loss over 100 epochs for Gather-scatter pattern detection on weighted temporal networks with 100K nodes. Imbalance $\approx 4.5\%$.

B.2 Scatter-gather

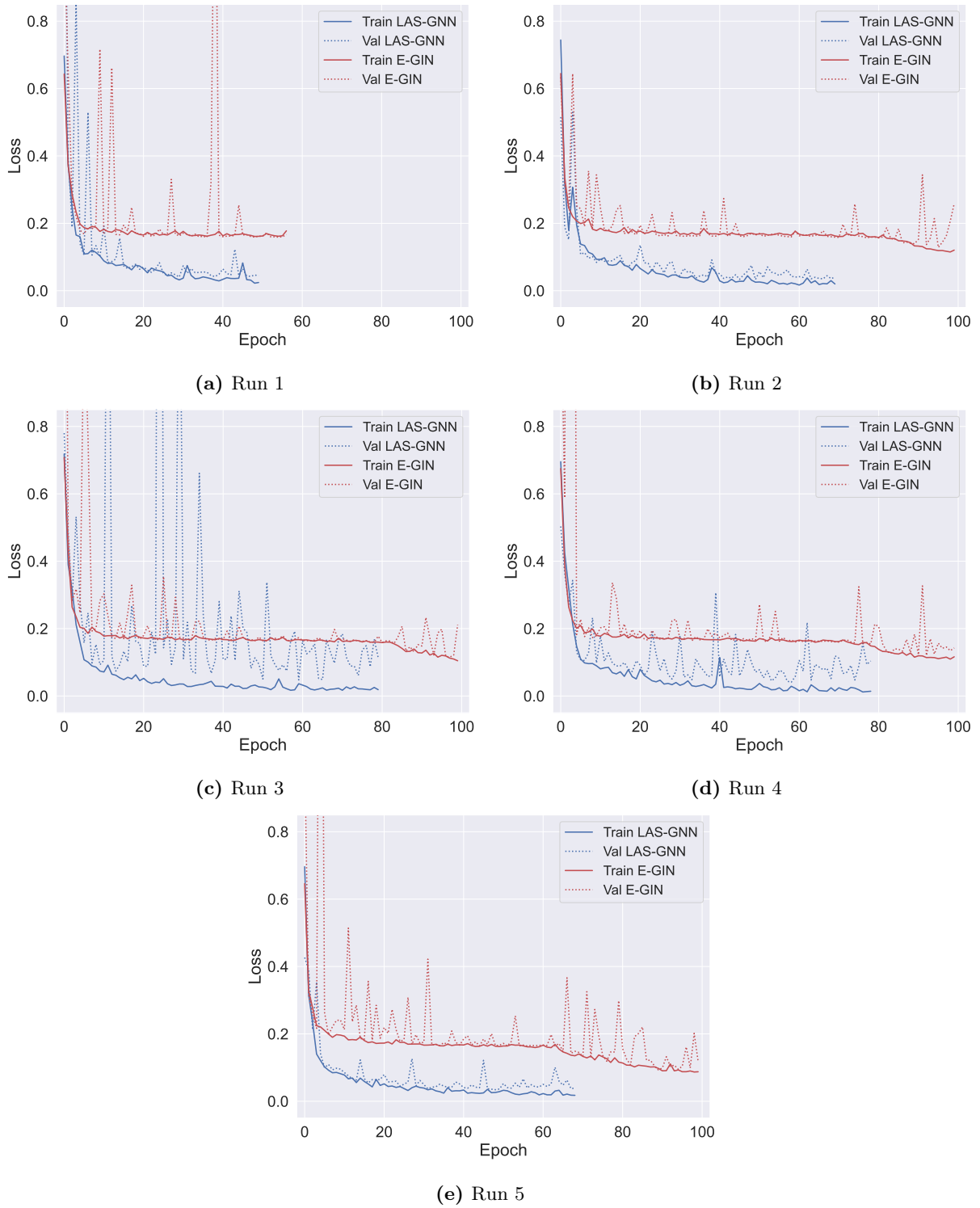


Figure B.2: Training and validation loss over 100 epochs for Scatter-gather pattern detection on weighted temporal networks with 100K nodes. Imbalance $\approx 3.5\%$.

B.3 C3

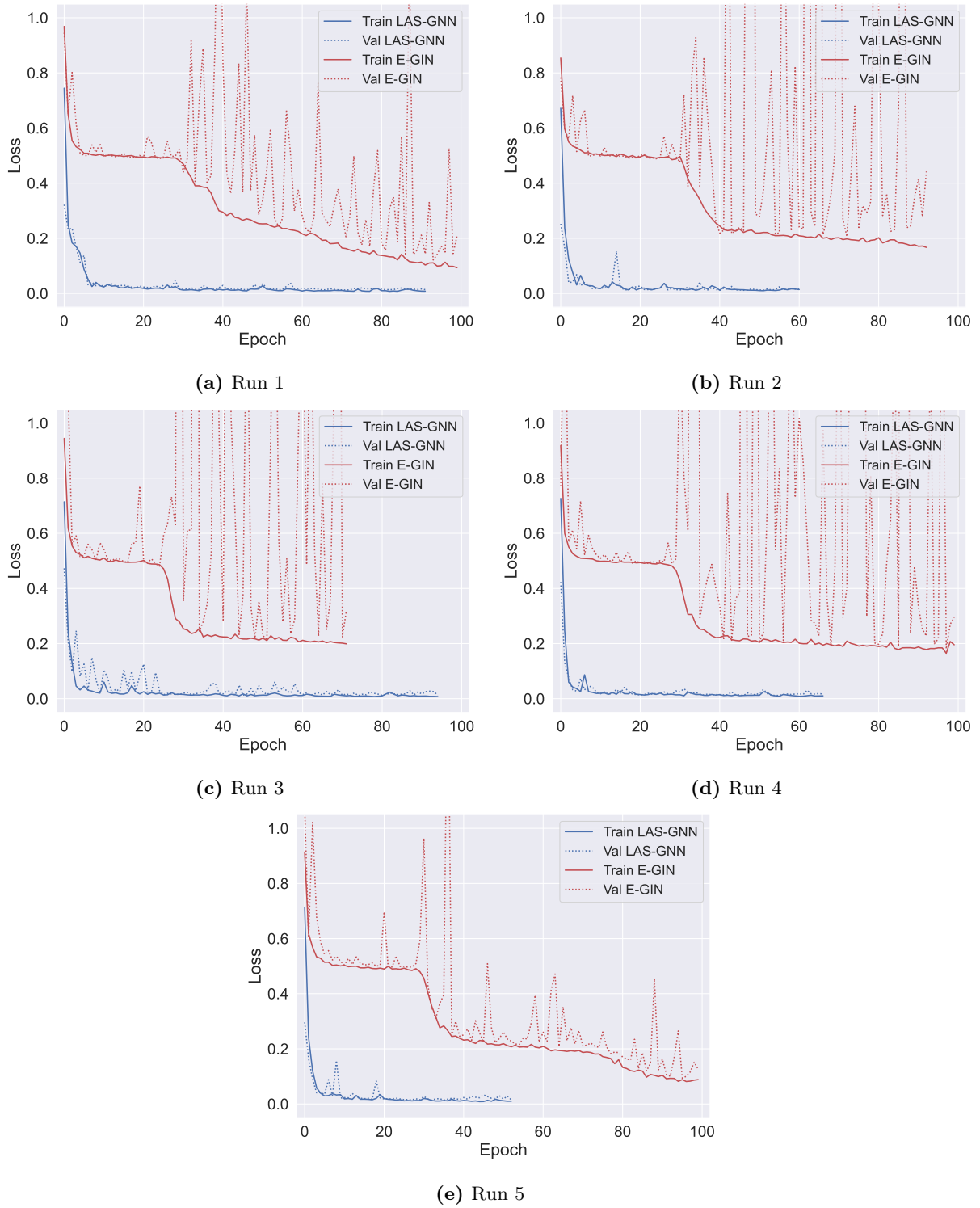


Figure B.3: Training and validation loss over 100 epochs for C3 pattern detection on weighted temporal networks with 100K nodes. Imbalance $\approx 5.1\%$.